AI 跃迁

JAVA BACKEND → AI APPLICATION ENGINEER

16 周知识复习册

从 Python、RAG、Agent 到 Java AI 工程。
112 节知识卡，按周整理，随时翻回。

16	112	3
周	节	个项目

打印建议：A4 纵向 · 默认边距 · 勾选背景图形

2026.10.26

CONTENTS

目录

W01 · Python 迁移与 AI 开发环境

D1 从 Java 到 Python：语法、容器与推导式 · D2 函数、类、类型与异常 · D3 工程化：uv、pytest、ruff 与配置 · D4 异步、HTTP 与第一个流式 LLM 调用 · D5 CLI 整合：配置、重试与成本 · D6 Lab：把 CLI 变成可演示作品 · D7 周复盘：Python 与 LLM 调用

W02 · FastAPI 与 AI 服务基础

D1 FastAPI 基础与分层 · D2 Pydantic v2 与错误处理 · D3 持久化：会话与消息 · D4 SSE 流式与韧性 · D5 测试、Docker 与交付 · D6 Lab：流式对话 API 完整版 · D7 周复盘：FastAPI 与 AI 服务

W03 · Transformer 与 LLM 推理直觉

D1 Tokenization：模型眼中的文字 · D2 Embedding 与相似度 · D3 Attention 与 Transformer Block · D4 预训练、SFT 与 RLHF · D5 解码、采样与成本 · D6 模型选型与评测 · D7 周复盘：LLM 原理

W04 · Prompt、结构化输出与工具调用

D1 Prompt 基础与模式 · D2 结构化输出与 JSON Schema · D3 Function Calling 与工具调用 · D4 Prompt Injection 与安全边界 · D5 评测与 LLM-as-judge · D6 Lab：JD-简历结构化匹配器 · D7 周复盘：Prompt 与工具调用

W05 · Embedding 与 RAG 基础

D1 Embedding 模型与向量库选型 · D2 文档解析与切分 · D3 向量检索、过滤与 MMR · D4 上下文拼装、引用与拒答 · D5 最小 RAG 评测 · D6 Lab：mini-rag 原型 · D7 周复盘：RAG 基础

W06 · 高级检索与 RAG 评测

D1 BM25 与混合检索 · D2 Rerank：用精度换延迟 · D3 Query 改写：多查询、HyDE 与分解 · D4 Parent-child、上下文压缩与引用对齐 · D5 RAGAS 与评测体系 · D6 Lab：优化版 RAG 与消融 · D7 周复盘：高级检索

W07 · 项目一（上）：企业知识库 RAG

D1 需求、架构与验收指标 · D2 Ingestion：上传、解析、切分、入库 · D3 索引与检索 · D4 问答、引用与拒答 · D5 多租户与权限 · D6 集成、端到端测试与 demo · D7 周复盘：项目一主链路

W08 · 项目一（下）：生产化与交付

D1 Docker Compose 与配置管理 · D2 缓存、限流与成本控制 · D3 可观测性：trace、token、延迟、错误 · D4 评测回归与 CI · D5 压测与性能优化 · D6 交付：README、架构图、视频、文章 · D7 周复盘 + 项目一模拟面试

W09 · Agent 基础与工具调用

D1 Agent 的适用边界 · D2 手写 ReAct 循环 · D3 工具设计、参数校验与幂等 · D4 Memory：短期、长期与遗忘 · D5 Guardrails、预算与红队 · D6 Lab：mini-agent 完整版 · D7 周复盘：Agent 基础

W10 · LangGraph 与工作流

D1 StateGraph：把 Agent 画成状态机 · D2 Checkpoint 与中断恢复 · D3 Human-in-the-loop：审批、修改、恢复 · D4 多智能体：supervisor 与专家 · D5 流式、可观测与任务评测 · D6 Lab：带审批的运营工作流 · D7 周复盘：LangGraph

W11 · 项目二：Agent 工作流平台

D1 需求、任务集与架构 · D2 工具集：搜索、SQL、知识库、文件 · D3 状态机、审批与恢复 · D4 记忆、权限与审计 · D5 Trace、评测与成本 · D6 集成、demo 与文档 · D7 周复盘 + 项目二模拟面试

W12 · 微调与数据工程

D1 微调决策树 · D2 数据工程：业务数据变训练样本 · D3 LoRA/QLoRA 原理与超参 · D4 训练实战 · D5 微调评测：对比 Prompt 基线 · D6 Lab：小模型微调与对比 · D7 周复盘：微调与数据

W13 · 推理部署与 Java AI 工程

D1 vLLM 与推理服务 · D2 量化与性能三角 · D3 Spring AI 基础 · D4 Spring AI RAG 与模型网关 · D5 Java 生产化：SSE、背压、熔断、指标 · D6 Lab：项目三完整版 · D7 周复盘 + 项目三模拟面试

W14 · AI 系统设计、ML 八股与算法

D1 系统设计框架与 RAG 设计 · D2 Agent 平台与模型网关设计 · D3 Transformer 与 ML 八股 · D4 算法模板：8 类高频模式 · D5 项目三系统设计与容量 · D6 全真模拟面试 1 · D7 周复盘：弱项补洞

W15 · 简历、项目包装与模拟面试

D1 简历 v3：一页、三条故事线、数字 · D2 三个项目故事卡：3/10/30 分钟版本 · D3 作品集与 demo · D4 模拟面试 2：系统设计 + 项目深挖 · D5 行为面试、转型故事与首批投递 · D6 春节：最低完成线 + 休息 · D7 恢复 + 模拟面试 3 + 投递复盘

W16 · 投递、面试迭代与 Offer

D1 定向投递 20 家 + 内推 · D2 面试复盘与弱项补洞 · D3 模拟面试 3 + 压力测试 · D4 系统设计与八股补洞 · D5 Offer 评估与谈判 · D6 16 周复盘与 90 天成长计划 · D7 毕业：保持手感，持续投递

用法：每个 Day 先看“一句话”和“核心知识”，遮住答案口述“面试口述”，再对照“常见坑”。会的打勾，不会的回到学习助手做随堂小测。

W01

Python 迁移与 AI 开发环境

7 节知识卡 · 先回忆，再对照

☐ DAY 1 从 Java 到 Python：语法、容器与推导式

Python 的容器、切片和推导式是内置表达式，先用它们消灭 Java 式 for 循环。

核心知识

- list 是动态数组：下标 O(1)，中间插入/删除 O(n)；dict 是哈希表：平均 O(1)，key 必须可哈希。
- tuple 不可变，适合做字典 key 或返回多值；set 用于去重和集合运算。
- 切片 [start:stop:step] 左闭右开，负索引从末尾算；切片会创建新对象（浅拷贝）。
- 推导式 [f(x) for x in xs if cond] 比手写 append 更清晰；dict/set 也有推导式。
- is 比较对象身份，== 比较值；小整数/字符串驻留会让 is 偶尔“看起来对”，但不要依赖。

再深入

- 机制：list 是连续数组，扩容时重新分配并均摊 O(1)；dict 是哈希表，冲突时探测，扩容会 rehash，所以最坏 O(n)。
- 边界：dict 只保证插入顺序，不保证有序遍历；自定义对象做 key 必须实现 __hash__/__eq__，参与哈希的字段要不可变。

常见坑

- 可变默认参数 def f(x, acc=[]) 会在多次调用间共享同一个 list；用 None 再创建。
- for i in range(len(xs)) 通常是 Java 思维；优先 for x in xs、enumerate、zip。

> **面试口述**
>
> Python 的 list 和 dict 底层分别是什么？为什么 dict 查找平均 O(1)?

追问

Q：list.sort() 和 sorted() 有什么区别？A：前者原地排序返回 None，后者返回新列表；都稳定，支持 key/reverse。

对应任务：学习舱 `w01d1`（4 个）

☐ DAY 2 函数、类、类型与异常

Python 用“协议 + 组合”替代大量继承，用 dataclass 和 typing 把数据结构写清楚。

核心知识

- 函数参数：位置参数、关键字参数、*args、**kwargs；默认参数在定义时求值一次。
- 闭包保存外层作用域变量；nonlocal 才能修改外层局部变量。
- @dataclass 自动生成 __init__/__repr__/__eq__；可变字段用 field(default_factory=list)。
- Protocol 是结构化子类型：只要方法签名匹配就满足，不需要显式继承。
- 异常要分层：业务异常、配置异常、外部依赖异常；用 raise … from e 保留因果链。

再深入

- 机制：Protocol 是结构化子类型，runtime_checkable 只检查属性是否存在，不校验签名；dataclass 生成 eq/repr，frozen=True 才不可变。
- 边界：异常不要静默吞掉；上下文管理器要覆盖异常路径；可变默认值必须用 field(default_factory=…)。

常见坑

- 类型注解不会在运行时强制校验；需要校验就用 Pydantic 或显式检查。
- 捕获 Exception 后不要静默 pass；至少记录上下文并决定是否重试。

> **面试口述**
>
> Python 的 Protocol 和 Java interface 有什么区别?

追问

Q：ABC 和 Protocol 怎么选？A：需要显式继承、共享实现用 ABC；解耦第三方实现、做 duck typing 用 Protocol。

对应任务：学习舱 `w01d2`（4 个）

☐ DAY 3 工程化：uv、pytest、ruff 与配置

让 Python 项目像 Java 项目一样可克隆、可测试、可重复构建。

核心知识

- pyproject.toml 是项目元数据与依赖的中心；uv.lock 锁定精确版本，保证可复现。
- uv sync 创建虚拟环境并安装依赖；uv run pytest 在项目环境中执行命令。
- pytest fixture 提供测试依赖；作用域 function/class/module/package/session 决定复用范围。
- parametrize 用数据驱动多个用例；tmp_path 提供隔离的临时目录。
- ruff 同时做格式化和 lint；pre-commit 在提交前自动检查。

再深入

- 机制：pyproject 声明依赖，uv.lock 固定解析结果；pytest fixture 用依赖注入管理测试资源，session 作用域只创建一次。
- 边界：session fixture 里的可变状态会跨用例污染；数据库测试可用事务回滚或 truncate，不要依赖执行顺序。

常见坑

- 测试之间共享全局状态/数据库会互相污染；用 fixture 建独立环境并在 teardown 清理。
- 只测“正常路径”不够；异常、超时、空输入、边界值才是 bug 高发区。

> **面试口述**
> pytest fixture 的作用域有哪些？怎么避免测试互相污染？

追问

Q：autouse fixture 有什么风险？A：隐式依赖、难追踪、可能拖慢测试；只在全局必要（如日志捕获）时使用。

对应任务：学习舱 `w01d3`（4 个）

☐ DAY 4 异步、HTTP 与第一个流式 LLM 调用

LLM 调用是典型的“高延迟 I/O”，异步和流式是体验与成本的关键。

核心知识

- async def 定义协程，await 让出事件循环；不要在异步函数里调用阻塞 SDK。
- asyncio.gather 并发执行多个协程；要配超时、取消和异常处理。
- httpx.AsyncClient 持有连接池，应复用；每个请求新建 client 会丢失连接复用。
- SSE 是单向 server→client 的文本协议：data: … 行 + 空行分隔事件。
- 上游错误要分类：401 不重试、429 退避、5xx 有限重试、超时取消。

再深入

- 机制：协程在 await 处让出事件循环，适合高并发 I/O；SSE 用 data 行和空行分隔事件，客户端逐事件消费。
- 边界：在 async 函数里调用阻塞 SDK 会卡住整个循环；流式 usage 可能只在最后一块返回；代理缓冲会让流式失效。

常见坑

- API key 不要写进代码或提交到 Git；用环境变量/密钥管理。
- 流式解析要处理半行、空行、[DONE] 和异常中断，不能假设每个 chunk 都是完整 JSON。

> **面试口述**
> SSE 和 WebSocket 的区别？为什么 LLM 流式常用 SSE？

追问

Q：asyncio.gather 中一个任务抛异常会怎样？A：默认向外传播，其他任务可能继续；需要 TaskGroup 或 return_exceptions 控制。

对应任务：学习舱 `w01d4`（4 个）

☐ DAY 5 CLI 整合：配置、重试与成本

把零散脚本组装成有配置、有日志、有预算的可演示工具。

核心知识

- 配置优先级：环境变量 > 配置文件 > 默认值；敏感信息只从环境变量读。
- CLI 参数用 argparse/typer；子命令和 --help 是可维护性的一部分。
- 重试只对可重试错误；指数退避 + 抖动，尊重 Retry-After。
- 会话历史要存全量，但发给模型时只取最近 N 轮 + 摘要。
- 成本 = 输入 token × 输入单价 + 输出 token × 输出单价；流式也要统计。

再深入

- 机制：配置优先级让部署环境覆盖默认值；重试通过退避把瞬时故障摊开，抖动避免大量客户端同时重试。
- 边界：非幂等操作不能盲重试；失败/重试请求也可能计费；用字符数估算 token 会严重低估成本。

常见坑

- 无限重试会把小故障放大成雪崩；必须设最大次数和总预算。
- 只统计“成功请求”的成本会低估真实花费；失败/重试也要计。

> **面试口述**
> 怎么设计一个可测试、可观测的 LLM 客户端？

追问

Q：重试和熔断的区别？A：重试处理瞬时故障；熔断在持续失败时快速失败，保护下游和调用方。

对应任务：学习舱 `w01d5`（3 个）

☐ DAY 6 Lab：把 CLI 变成可演示作品

可运行、可解释、可复现，比功能多更重要。

核心知识

- 项目结构：入口层（CLI）→ 配置层 → 客户端适配层 → 会话/存储层。
- 适配器模式隔离不同模型供应商；业务逻辑不直接依赖某家 SDK。
- 上下文溢出策略：滑动窗口、摘要、检索相关历史、分层记忆。
- 测试用 mock transport 构造普通响应、流式 chunk、超时和错误。
- 演示脚本：问题 → 方案 → 现场操作 → 结果 → 限制，控制在 3 分钟。

再深入

- 机制：Adapter 隔离供应商差异；上下文管理是 token 预算下的信息选择问题，不是简单截断。
- 边界：摘要会丢细节，必须结构化保留用户偏好、硬约束和未完成事项；mock 测试稳定，但不能替代真实 API 冒烟。

常见坑

- 摘要会丢细节；用户偏好、硬约束、未完成事项要结构化保留。
- 演示时不要从安装依赖开始讲；先让观众看到问题和结果。

> **面试口述**
> 对话历史超过上下文窗口时，你怎么处理？

追问

Q：摘要应该保留什么？A：硬约束、偏好、关键事实、未完成事项和下一步；不要保留客套和重复内容。

对应任务：学习舱 `w01d6`（4 个）

☐ DAY 7 周复盘：Python 与 LLM 调用

把“我学过”变成“我能讲清、能复现、能改进”。

核心知识

- 复盘三问：哪些任务真正完成了？哪个概念仍讲不清？下周先修哪一个？
- 错题不是收藏，而是要在 24 小时内重做一遍并写原因。
- 项目介绍结构：背景 → 约束 → 方案 → 取舍 → 指标 → 失败 → 如果重做。
- Python 迁移的验收：能读、能写、能测试、能排障，而不是背语法。

再深入

- 机制：主动回忆比重复阅读更有效；错题重做暴露的是提取失败，而不是“看没看过”。
- 边界：只看不写、只收藏错题不重做，都会产生“学会了”的错觉；项目叙事没有数据就无法证明。

常见坑

- 只看视频不写代码，下周会全部忘掉；输入必须转化为 commit。
- 周测低于 60% 不是失败，是下周的第一个任务。

> **面试口述**
>
> 用 2 分钟介绍 llm-cli 的架构和取舍。

追问

Q：怎么证明这一周有进步？A：同一组题的正确率、错题复现率、口述完整度、代码 commit 和项目指标，至少两项可对比。

对应任务：学习舱 `w01d7`（4 个）

W02

FastAPI 与 AI 服务基础

7 节知识卡 · 先回忆，再对照

☐ DAY 1 FastAPI 基础与分层

路由只负责协议，业务放 service，外部依赖通过依赖注入隔离。

核心知识

- FastAPI 用类型注解生成校验和 OpenAPI 文档；请求体/查询/路径参数都有明确来源。
- Depends 默认在同一请求内缓存依赖结果，适合数据库 session、当前用户等请求级资源。
- lifespan 管理启动/关闭资源（连接池、HTTP client），不要在每个请求里重复创建。
- 统一异常处理：业务异常映射成稳定错误码，不要把内部堆栈返回给客户端。
- 分层：router（协议）→ service（业务）→ repository/client（外部依赖）。

再深入

- 机制：FastAPI 用依赖图解析参数，Depends 默认请求级缓存；lifespan 管理进程级资源，路由只做协议转换。
- 边界：Depends 不是全局单例；在异步路由里调用阻塞库会卡事件循环；业务逻辑塞进 router 会难测试。

常见坑

- 在路由里直接写 SDK 调用，导致难测试、难复用；业务逻辑要能脱离 HTTP 运行。
- 在异步路由里调用阻塞库，会卡住整个事件循环；阻塞操作放线程池或改异步客户端。

> **面试口述**
>
> FastAPI 的依赖注入和 Spring 的 DI 有什么异同?

追问

Q：测试时怎么替换依赖？A：用 app.dependency_overrides 覆盖 provider，测试结束清理。

对应任务：学习舱 `w02d1`（4 个）

☐ DAY 2 Pydantic v2 与错误处理

用模型定义契约，把“不可信输入”挡在业务层之外。

核心知识

- BaseModel 做解析、类型转换和校验；Field 限制范围、长度、默认值。
- field_validator 校验单字段，model_validator 校验跨字段；分 before/after 模式。
- model_config 控制额外字段、别名、序列化；pydantic-settings 读环境变量。
- 校验失败 FastAPI 默认返回 422；业务错误用自定义异常 + 统一响应体。
- 响应模型要显式声明，避免把内部字段（密码、token）意外返回。

再深入

- 机制：Pydantic 先解析再校验，field validator 管单字段，model validator 管跨字段；response_model 控制输出白名单。
- 边界：校验失败默认 422，不要和业务 400 混用；extra 字段是否允许要显式配置；strict 模式会改变类型转换行为。

常见坑

- 只做前端校验，不做后端校验；API 可以被任意客户端直接调用。
- 错误响应格式一会一个样，客户端无法稳定处理；错误码要版本化、可机读。

> **面试口述**
>
> field validator 和 model validator 的区别？什么时候用哪个?

追问

Q：field validator 和 model validator 怎么选？A：单字段约束用 field，跨字段/整模型用 model；都分 before/after。

对应任务：学习舱 `w02d2`（4 个）

☐ DAY 3 持久化：会话与消息

存全量消息，发给模型时只取需要的部分。

核心知识

- SQLAlchemy 2.0 用 AsyncSession；session 不是并发安全的，按请求/任务创建。
- 表：conversations（会话）与 messages（消息），外键 + 索引；软删除保留审计。
- 消息表存 role/content/token/model/latency/created_at，支持成本、审计、重放。
- 事务边界放在 service：一次业务操作要么全成功，要么回滚。
- 上下文裁剪策略：最近 N 轮 + 摘要 + 与当前问题相关的历史。

再深入

- 机制：AsyncSession 绑定连接和事务上下文；repository 封装查询，service 控制事务边界。
- 边界：Session 不能跨并发任务共享；懒加载在异步下可能报错；深分页 offset 会越来越慢。

常见坑

- 把完整历史每轮都塞回模型，token 成本会指数级增长。
- 在多个协程间共享同一个 AsyncSession，会出现事务串线/连接错误。

> **面试口述**
> 为什么消息表要存 token、模型和耗时？除了“以后可能有用”还有什么价值？

追问

Q：offset 和 cursor 分页怎么选？A：后台小数据可用 offset；面向用户、数据量大、要求稳定用 cursor。

对应任务：学习舱 `w02d3`（3 个）

☐ DAY 4 SSE 流式与韧性

流式不是“把响应写出去”，而是处理断连、超时、上游失败和背压。

核心知识

- StreamingResponse + text/event-stream；每个事件以空行结束，客户端用 EventSource/fetch 读取。
- 客户端断开要捕获取消信号，取消上游请求、释放连接、记录部分 usage。
- 代理/网关默认可能缓冲响应，必须关闭 buffering，否则“流式”变一次性。
- 429 用退避 + 抖动，尊重 Retry-After；5xx 有限重试；401 不重试。
- 每个请求带 request id，日志、trace、错误响应统一关联。

再深入

- 机制：SSE 是长连接上的事件流；StreamingResponse 逐块写出，客户端断开通过取消信号向上游传播。
- 边界：没有心跳的连接可能被中间层断开；代理缓冲会把流式变成一次性；客户端断开后继续生成会浪费成本。

常见坑

- 断连后后台还继续生成，既浪费成本又可能写脏数据。
- 把 SSE 当成 WebSocket 用，试图客户端→服务端双向通信。

> **面试口述**
> 客户端断开流式连接时，服务端应该做哪些清理？

追问

Q：怎么测试流式接口？A：httpx stream 读取多块，断言事件顺序、[DONE]、断连清理和错误事件。

对应任务：学习舱 `w02d4`（3 个）

☐ DAY 5 测试、Docker 与交付

新机器能一键启动，才算真正的交付。

核心知识

- pytest + httpx AsyncClient 测 API；依赖覆盖替换数据库/上游，保证测试稳定。
- 覆盖正常、校验失败、上游错误、流式、断连五类路径。
- Docker 多阶段构建：构建阶段装依赖，运行阶段只拷产物；用 slim 镜像。
- 非 root 运行、.dockerignore、健康检查、环境变量注入密钥。
- README 写清启动命令、环境变量、健康检查、已知限制。

再深入

- 机制：多阶段构建把编译依赖和运行时分离；健康检查让编排系统知道服务是否真正可用。
- 边界：镜像越小不一定启动越快；构建缓存会影响层复用；密钥不能进入镜像层。

常见坑

- 镜像里包含 .env、测试数据、Git 历史；用 .dockerignore 和最小上下文。
- 只测 healthcheck，不测真实业务链路；上线后才发现参数校验/流式有问题。

> **面试口述**
> Docker 镜像怎么减小？多阶段构建具体省掉了什么？

追问

Q：Testcontainers 和 mock 怎么选？A：集成/契约测试用 Testcontainers，单元测试用 mock；两者都要有。

对应任务：学习舱 `w02d5`（3 个）

☐ DAY 6 Lab：流式对话 API 完整版

并发流式最怕共享可变状态；隔离、限流、可观测是三件套。

核心知识

- 每个请求独立上下文/生成器，不共享 buffer；request id 贯穿日志和响应。
- 限流保护上游配额和成本；按用户/租户/模型维度分别限。
- 压测看首 token、P95、错误率、并发连接，不只看 QPS。
- 连接池大小、上游配额、数据库连接数通常先成为瓶颈。
- 优雅关闭：停止接收新请求，等待进行中的流结束或超时。

再深入

- 机制：每个流式请求需要独立上下文和生成器；限流保护上游配额，连接池限制数据库并发。
- 边界：共享可变 buffer 会串消息；连接池耗尽会让请求排队；上游 429 会放大延迟。

常见坑

- 20 路并发不串消息的前提是“无共享可变 buffer”，不是加锁就能解决。
- 压测只打 healthcheck，没覆盖真实模型调用，数据没有意义。

> **面试口述**
> 20 路并发流式请求，怎么保证不串消息？

追问

Q：为什么只看平均延迟不够？A：平均值掩盖长尾；流式体验由 P95/P99 和错误率决定。

对应任务：学习舱 `w02d6`（4 个）

☐ DAY 7 周复盘：FastAPI 与 AI 服务

服务化能力 = 协议 + 校验 + 持久化 + 韧性 + 可交付。

核心知识

- 复盘 API 的五个边界：输入、输出、超时、错误、权限。
- 能画出请求生命周期：网关 → 路由 → 依赖 → service → 上游 → 持久化 → 流式返回。
- 能解释一次 429/断连/上游超时的完整处理链路。
- 能说清 Docker 镜像里有什么、密钥从哪里来。

再深入

- 机制：SLO 是可用性和延迟的目标，错误预算决定什么时候该停止加功能、先修稳定性。
- 边界：只测 happy path 会漏掉真实故障；日志里打印完整文档/用户输入有合规风险。

常见坑

- 只关注“接口能返回”，忽略超时、重试、限流，线上会被上游拖死。
- 把配置写死在代码里，换环境就要改代码。

> **面试口述**
>
> 用 2 分钟讲 chat-api 的架构、并发模型和故障处理。

追问

Q：一次请求至少记录什么？A：request id、租户、模型、token、首 token/总延迟、错误码、是否降级。

对应任务：学习舱 `w02d7`（4 个）

W03

Transformer 与 LLM 推理直觉

7 节知识卡 · 先回忆，再对照

☐ DAY 1 Tokenization：模型眼中的文字

模型不读字符，它读 token；token 数直接决定成本、上下文和延迟。

核心知识

- BPE 从字符开始，反复合并高频相邻片段，形成子词词表；WordPiece/SentencePiece 思路类似但细节不同。
- 中文 token 效率取决于词表和训练语料；同一句话在不同模型上 token 数可能差很多。
- 特殊 token（BOS/EOS/pad/system）参与对话模板；模板不一致会影响效果。
- 上下文窗口按 token 计，不是按字数；长文档要切分或压缩。
- 成本按输入/输出 token 分别计费，输出通常更贵。

再深入

- 机制：BPE 从字符开始合并高频片段，词表大小和合并规则决定切分；特殊 token 参与对话模板。
- 边界：token 数不等于字符数；同一句话在不同模型上不同；长上下文按 token 计费。

常见坑

- 用“1 个汉字 = 1 个 token”估算成本，实际可能差 1.5–2 倍。
- 只算输入 token，忽略输出和重试；真实账单会高很多。

> **面试口述**
>
> 为什么同一个中文句子在不同模型上 token 数不同？这会带来什么工程影响？

追问

Q：怎么降低 token 成本？A：裁剪上下文、摘要、缓存、批量、选更高效词表的模型，并监控输出 token。

对应任务：学习舱 w03d1（4 个）

☐ DAY 2 Embedding 与相似度

embedding 把语义压成向量，检索就是在向量空间里找“最近的意思”。

核心知识

- embedding 模型把文本映射到固定维度向量；语义相近的文本距离更近。
- 余弦相似度看夹角，点积看方向和模长；向量归一化后两者排序等价。
- 维度、语言、领域、最大输入长度都会影响模型选择；MTEB 可参考但不能代替业务评测。
- 相似不等于相关：同义句可能很近，反义句也可能很近（取决于模型）。
- 检索失败常见原因：切分切断语义、模型不适配领域、query 太短、距离度量不匹配。

再深入

- 机制：embedding 把文本映射到同一向量空间，余弦看夹角；归一化后点积与余弦排序等价。
- 边界：不同模型向量不可混用；短 query 与长文档语义密度不同；领域词可能不在模型分布内。

常见坑

- 把 embedding 当“关键词匹配”，忽略 query 改写和混合检索。
- 不做归一化就直接比较点积，长文本会占便宜。

> **面试口述**
>
> 余弦相似度和点积什么时候等价？为什么很多模型建议先归一化？

追问

Q：什么时候需要微调 embedding？A：领域术语多、召回长期不达标、有标注数据时；先试更好的通用模型和混合检索。

对应任务：学习舱 w03d2（4 个）

☐ DAY 3 Attention 与 Transformer Block

self-attention 让每个 token 根据相关性聚合其他 token 的信息。

核心知识

- Q/K/V：Q 与 K 点积得到相关性权重，softmax 归一化后加权聚合 V。
- 缩放因子 sqrt(d_k) 控制点积方差，避免 softmax 饱和和梯度消失。
- 多头让模型在不同子空间关注不同关系（语法、指代、位置等），最后拼接。
- 位置编码提供顺序信息；残差 + LayerNorm 让深层网络稳定训练。
- FFN 做逐 token 非线性变换；encoder/decoder 结构决定注意力可见范围。

再深入

- 机制：Q 与 K 点积得到相关性，softmax 归一化后加权 V；多头在不同子空间学习不同关系。
- 边界：标准 attention 是 $O(n^2)$；attention 权重不是因果解释；pre-norm/post-norm 影响训练稳定性。

常见坑

- 只背“QKV”，讲不清“每个词如何看其他词”；面试官一追问就露馅。
- 把 attention 权重直接当“解释”，它只是相关性，不是因果。

> **面试口述**
> self-attention 为什么要除以 sqrt(d_k)？多头的作用是什么？

追问

Q：为什么除以 sqrt(d_k)？A：控制点积方差，避免 softmax 饱和和梯度消失。

对应任务：学习舱 w03d3（4 个）

☐ DAY 4 预训练、SFT 与 RLHF

预训练学语言，SFT 学指令，RLHF/DPO 学偏好。

核心知识

- 预训练：海量无标注文本，目标是下一个 token 预测；得到基座模型。
- SFT：指令-回答对，教模型跟随指令、输出格式；数据质量决定上限。
- RLHF：用人类偏好训练奖励模型，再用 RL 优化；DPO 直接用偏好对优化，工程更简单。
- 基座模型不等于助手模型；instruct 版本经过后训练，更会“听话”。
- 灾难性遗忘：窄领域微调可能损害通用能力，要混入通用数据并做回归。

再深入

- 机制：预训练学下一个 token，SFT 学指令-回答映射，RLHF/DPO 用偏好优化有用性和安全性。
- 边界：SFT 数据质量决定上限；偏好数据昂贵；窄领域微调可能灾难性遗忘。

常见坑

- 以为什么都能靠微调解决；知识更新、权限、引用更适合 RAG。
- SFT 数据只追求数量，格式不统一、答案错误，模型会学坏。

> **面试口述**
> SFT 和 RLHF 分别解决什么问题？为什么不直接上 RLHF？

追问

Q：DPO 和 PPO 的区别？A：DPO 直接用偏好对优化，工程简单；PPO 需要奖励模型和 RL 训练，控制更复杂。

对应任务：学习舱 w03d4（4 个）

☐ DAY 5 解码、采样与成本

模型输出的是概率分布，采样策略决定“稳定”还是“多样”。

核心知识

- greedy 每步选最高概率；beam search 保留多个候选，适合翻译等任务。
- temperature 缩放 logits：越低越稳定，越高越发散。
- top-k 只保留概率最高的 k 个；top-p 保留累计概率达到 p 的最小集合。
- stop 控制停止词；max_tokens 控制成本上限；seed 不保证跨平台完全复现。
- temperature=0 也不保证确定性：浮点、批处理、硬件、模型版本都会带来差异。

再深入

- 机制：模型输出是概率分布，temperature 缩放 logits，top-p 截断累计概率，stop/max_tokens 控制终止和成本。
- 边界：temperature=0 也不保证跨平台复现；temperature 和 top-p 同时大改无法归因；输出 token 通常更贵。

常见坑

- 同时大改 temperature 和 top_p，无法归因；一次只调一个。
- 把 temperature=0 当成“可复现实验”，线上仍可能漂移。

> **面试口述**
> temperature 和 top-p 的区别？事实型任务怎么配？

追问

Q：logprobs 有什么用？A：看模型置信度、做分类/打分、排查异常输出，但不等于事实正确。

对应任务：学习舱 w03d5（4 个）

☐ DAY 6 模型选型与评测

选模型不是选“最强”，而是在质量、延迟、成本、合规之间找约束最优。

核心知识

- 先定质量门槛和 SLO，再比较模型；没有约束的选型没有意义。
- 小模型做基线，大模型看上限；用固定评测集比较，不看排行榜单。
- 评测维度：正确性、格式合法率、忠实度、延迟、成本、稳定性。
- 本地模型收益是隐私/成本/可控，代价是质量、运维和硬件。
- 供应商锁定、限流、版本变更、数据合规都要纳入决策。

再深入

- 机制：模型选型是约束优化：质量门槛、P95、成本、上下文、部署方式、合规。
- 边界：排行榜不等于业务效果；模型版本更新会漂移；本地模型省 API 费但增加运维和硬件成本。

常见坑

- 只看几个“顺眼”的样例就选型；没有评测集，无法复现。
- 忽略模型版本更新导致的输出漂移。

> **面试口述**
> 怎么为一个业务任务选模型？你会用哪些指标做决策？

追问

Q：怎么评估一个模型是否适合业务？A：固定任务集 + 人工抽检 + 延迟/成本/稳定性，和 baseline 对比。

对应任务：学习舱 w03d6（4 个）

□ DAY 7 周复盘：LLM 原理

从 token 讲到生成，能把每一步的取舍讲清楚，就够面试用了。

核心知识

- 一条主线：tokenizer → embedding → Transformer → logits → 采样 → 自回归。
- 三个工程约束：上下文窗口、延迟、成本。
- 三种后训练：SFT、RLHF、DPO；知道各自目标即可。
- 幻觉的根因是“预测下一个 token”，RAG/引用/拒答是缓解，不是根治。

再深入

- 机制：整条链路是 tokenizer → embedding → Transformer → logits → 采样 → 自回归；每一环都有成本和误差。
- 边界：RAG 缓解幻觉但不根除；上下文窗口有限；attention 不是数据库查询。

常见坑

- 把“参数量大”当成唯一优势，忽略延迟和成本。
- 背论文名词但讲不出工程含义。

> **面试口述**
>
> 用 3 分钟从 token 讲到生成。

追问

Q：temperature=0 为什么还会不稳定？A：浮点、批处理、硬件、模型版本和外部检索结果都会变化。

对应任务：学习舱 `w03d7`（4 个）

W04

Prompt、结构化输出与工具调用

7 节知识卡 · 先回忆，再对照

☐ DAY 1 Prompt 基础与模式

好 prompt 不是“请一步步思考”，而是把任务、约束、输入、输出格式说清楚。

核心知识

- 五要素：角色/任务、输入数据、约束、输出格式、示例。
- zero-shot 适合通用任务；few-shot 用 3–5 个覆盖边界和难例的示例。
- CoT 对复杂推理有帮助，但会增加 token、可能泄露推理；简单任务不必默认开启。
- 指令冲突会让模型无所适从；系统指令、用户指令、数据要分层标记。
- prompt 要版本化：版本、改动、评测结果、失败案例都要记录。

再深入

- 机制：prompt 是任务契约：角色、输入、约束、输出格式、示例；系统/用户/数据分层，减少指令冲突。
- 边界：few-shot 示例过多会占上下文、过拟合示例风格；CoT 会增加 token，简单任务可能变差。

常见坑

- 一次塞入多个互相冲突的目标（既要简短又要详尽），模型只能猜。
- 只改 prompt 不看评测，越改越“感觉良好”。

> **面试口述**
> few-shot 的示例怎么选？示例越多越好吗？

追问

Q：怎么管理 prompt 版本？A：版本号 + 改动说明 + 固定评测集 + 失败案例，支持回滚。

对应任务：学习舱 `w04d1`（4 个）

☐ DAY 2 结构化输出与 JSON Schema

让模型输出 JSON 只是第一步，能校验、能重试、能降级才是工程。

核心知识

- JSON Schema 定义字段、类型、枚举、必填、范围；Pydantic 可在应用侧二次校验。
- JSON mode 只保证语法上是 JSON，不保证字段和语义正确。
- 解析失败时把错误信息带回模型重试；仍失败则降级为文本 + 人工处理。
- 让模型输出 evidence 字段，把结论和原文绑定，降低胡编。
- 流式结构化输出要处理半截 JSON，通常先累积再解析，或使用增量解析器。

再深入

- 机制：JSON Schema/JSON mode 约束语法，Pydantic 在应用侧做二次校验；解析失败要带错误重试。
- 边界：JSON mode 不保证字段语义正确；流式 JSON 可能只有半截；重试会增加成本和延迟。

常见坑

- 用正则/eval 硬修 JSON；eval 有安全风险，正则修不好嵌套结构。
- schema 太复杂，模型字段合法率下降；先简化，再逐步加约束。

> **面试口述**
> 模型输出的 JSON 解析失败，你怎么处理？为什么不能直接 eval?

追问

Q：结构化输出和 function calling 怎么选？A：只要数据用 schema；要执行动作/查数据用 tool calling。

对应任务：学习舱 `w04d2`（4 个）

☐ DAY 3 Function Calling 与工具调用

模型只输出“调用意图”，真正执行、校验和负责的是应用。

核心知识

- tool schema 包含名称、描述、参数 JSON Schema；描述要写清何时用、何时不用。
- 模型返回 tool name + arguments；应用校验参数、执行、把结果作为 observation 回传。
- 并行工具调用可以减少轮次，但要注意依赖关系和结果顺序。
- 工具错误要结构化返回（类型/原因/是否可重试），让模型能决定重试/换工具/终止。
- 高风险工具（写库、发通知、执行 SQL）要加权限、审批、幂等和审计。

再深入

- 机制：模型输出 tool name + arguments，应用校验、执行、回传 observation；模型不直接拥有执行权限。
- 边界：工具描述含糊会选错；并行调用有依赖顺序；写操作要幂等、审批、审计。

常见坑

- 让模型直接拼 SQL/shell；必须参数化、白名单、最小权限。
- 工具描述含糊，模型选错工具；描述本身就是 prompt 的一部分。

> **面试口述**
> function calling 是模型执行函数吗？权限和幂等应该放在哪一层？

追问

Q：并行工具调用要注意什么？A：只并行无依赖的只读工具；结果要按 id 对齐，失败要单独处理。

对应任务：学习舱 `w04d3`（4 个）

☐ DAY 4 Prompt Injection 与安全边界

把检索到的文档、网页、工具结果都当成“不可信输入”，而不是指令。

核心知识

- 直接注入：用户在输入里写“忽略以上指令”；间接注入：恶意内容藏在文档/网页/工具结果里。
- 模型难以可靠区分“指令”和“数据”，所以不能只靠 prompt 防御。
- 纵深防御：最小权限、工具白名单、输入隔离/标记、输出校验、人工审批、审计。
- 系统提示泄露、数据外泄、越权工具调用是三类高频风险。
- 安全测试要进入回归集，不能上线前才做一次。

再深入

- 机制：注入的本质是让不可信数据获得指令权；直接注入在用户输入，间接注入在文档/网页/工具结果。
- 边界：prompt 里的“不要听用户的”只是软约束；模型无法可靠区分指令和数据。

常见坑

- 在 prompt 里写“不要听用户的”就以为安全；攻击者可以间接注入。
- 只防输入不防输出；模型可能泄露系统提示或内部数据。

> **面试口述**
> 为什么“在 prompt 里写不要听用户的”不够？你会怎么设计防御？

追问

Q：怎么测试注入防御？A：固定攻击集，覆盖直接/间接/越权/泄露，纳入回归并记录残余风险。

对应任务：学习舱 `w04d4`（4 个）

☐ DAY 5 评测与 LLM-as-judge

没有评测集的 prompt 优化都是玄学。

核心知识

- 评测集结构：输入、参考答案、评分 rubric、相关证据；覆盖简单/边界/无答案/对抗样本。
- 指标：字段合法率、准确率、忠实度、引用命中、延迟、成本。
- LLM-as-judge 有位置偏差、长度偏好、自我偏好、对 prompt 敏感等问题。
- 用人工标注校准 judge；固定 rubric；交换顺序/双盲；抽检。
- 保留 held-out 集，调参不能反复看它，否则会过拟合。

再深入

- 机制：评测集是回归测试，rubric 定义什么叫好；LLM judge 只是近似，不是金标准。
- 边界：模型自出题、自评分会虚高；judge 有位置/长度/自我偏好；held-out 集不能反复调参。

常见坑

- 用模型生成评测题，再让同一个模型评分；自我循环，分数虚高。
- 只看平均分，不看失败分布；长尾错误可能更致命。

> **面试口述**
> LLM-as-judge 有什么风险？你怎么校准它？

追问

Q：怎么校准 judge？A：人工标注一批样本，固定 rubric，交换顺序/双盲，比较 judge 与人工一致性。

对应任务：学习舱 `w04d5`（4 个）

☐ DAY 6 Lab：JD - 简历结构化匹配器

把 prompt、schema、工具、评测串成一个小产品，而不是一堆脚本。

核心知识

- 输入清洗：JD/简历去噪、分段、长度控制；敏感信息脱敏。
- 输出契约：匹配分、优势、缺口、证据、下一步建议。
- 证据字段要引用原文片段，便于人工复核和自动校验。
- 批量处理要控制并发、限流、重试和成本。
- README 要写清适用边界和失败案例，主动暴露局限。

再深入

- 机制：匹配器 = 输入清洗 → prompt/schema → LLM → 校验/重试 → 证据与报告；批量处理要控制并发和预算。
- 边界：评分 rubric 不清会导致不可复现；简历含 PII；只展示成功案例不可信。

常见坑

- 只展示“匹配成功”的样例；面试官会问失败案例。
- 评分没有 rubric，10 个人看出 10 个分数。

> **面试口述**
> 这个项目怎么防止“看起来很准”？你用什么指标证明它有效？

追问

Q：怎么证明匹配器有效？A：固定标注集、评分相关性、证据命中率、baseline 对比和失败案例。

对应任务：学习舱 `w04d6`（4 个）

□ DAY 7 周复盘：Prompt 与工具调用

Prompt 是接口契约，评测是回归测试，安全是上线前提。

核心知识

- 能解释 prompt 五要素和 CoT 的适用边界。
- 能设计 JSON Schema、处理解析失败和降级。
- 能说清 function calling 的责任边界和工具安全。
- 能列出 prompt injection 的至少 5 种防御手段。

再深入

- 机制：prompt 工程是“契约 + 版本 + 评测 + 安全”的闭环，不是一句咒语。
- 边界：没有评测的优化是主观感觉；prompt 变更可能让安全测试失效；模型升级会漂移。

常见坑

- 把“模型很聪明”当工程方案；边界、失败、成本才是工程。
- 安全只在最后补，结果要重构整个工具层。

> **面试口述**
>
> 用 2 分钟讲你怎么把 prompt 工程化。

追问

Q：prompt 回归应该看什么？A：字段合法率、任务指标、忠实度、延迟、成本和安全用例通过率。

对应任务：学习舱 `w04d7`（4 个）

W05 Embedding 与 RAG 基础

7 节知识卡 · 先回忆，再对照

☐ DAY 1 Embedding 模型与向量库选型

先选“够用且可运维”的组合，再谈性能极限。

核心知识

- 选 embedding：语言/领域、维度、最大输入长度、成本、是否可本地部署。
- pgvector 适合已有 Postgres 的团队：事务、权限、metadata filter 都能复用。
- 专用向量库（Qdrant/Milvus）在超大规模、复杂过滤、分布式上更强，但多一套运维。
- HNSW 参数：m 连接数、ef_construction 建索引候选、ef_search 查询候选；越大召回越高、资源越多。
- 距离函数要与模型训练时一致（cosine/inner product/L2）。

再深入

- 机制：HNSW 用多层图做近似最近邻，上层稀疏负责快速跳转，下层稠密负责精细搜索；m 决定连接度，ef 决定候选集。
- 边界：ef_search 越大召回越高但延迟越大；m 越大内存越多；过滤条件太多时 HNSW 可能退化为扫描。

常见坑

- 只看 benchmark 选模型，不看中文/领域/长文本表现。
- 距离函数和索引 opclass 不匹配，召回结果会很奇怪。

> **面试口述**
>
> pgvector 的 HNSW 参数 m、ef_construction、ef_search 分别影响什么？

追问

Q：HNSW 和 IVF 怎么选？A：HNSW 查询快、召回好但内存高；IVF 更省内存、适合大批量，需调 nlist/nprobe。

对应任务：学习舱 `w05d1`（4 个）

☐ DAY 2 文档解析与切分

切分是 RAG 最容易被低估、但收益最大的环节。

核心知识

- 解析要保留结构：标题、段落、表格、页码、来源；这些是 metadata 和引用的基础。
- 切分策略：固定窗口、递归分隔符、按标题/段落、语义切分、parent-child。
- chunk 太小丢上下文，太大稀释 embedding、增加噪声和成本；常见 300–800 token，重叠 10–20%。
- 中文按标点/段落切；代码按函数/类切；表格单独处理。
- metadata：tenant、doc_id、version、title、page、section、chunk_index。

再深入

- 机制：解析保留结构，切分把长文档变成可检索单元；chunk 是检索和生成的共同上下文单位。
- 边界：固定窗口会切断句子/表格；chunk 太大稀释 embedding；metadata 缺失会导致无法过滤和引用。

常见坑

- 直接按固定字符切，把一句话/表格切断，召回后模型也读不懂。
- 不存原文位置，引用无法定位到页码/段落。

> **面试口述**
>
> 为什么按标题/段落切分通常比固定窗口好？什么时候反而适合固定窗口？

追问

Q：chunk size 怎么定？A：先按结构切，再用评测集做 300/500/800 token 消融，同时看召回、忠实度和成本。

对应任务：学习舱 `w05d2`（4 个）

☐ DAY 3 向量检索、过滤与 MMR

top-k 不是越大越好；先保证“能召回”，再保证“不重复、不越权”。

核心知识

- 相似度检索返回 top-k；k 增大召回可能上升，但噪声、延迟、成本也上升。
- metadata filter 必须下推到检索层，不能先全库召回再在应用层过滤。
- MMR 在相关性和多样性之间权衡，减少重复 chunk；lambda 越接近 1 越重相关性。
- 多租户/权限过滤要进入查询条件，缓存 key 也要带租户。
- 召回失败先看：query、切分、embedding、filter、索引参数。

再深入

- 机制：向量检索返回 top-k，filter 缩小候选空间，MMR 在相关性和多样性间做贪心选择。
- 边界：先全库召回再过滤会越权且召回不足；k 太大会引入噪声；MMR 会增加计算。

常见坑

- 为了“召回更多”把 k 调到 100，结果上下文塞满噪声，答案更差。
- 只在应用层过滤租户，导致越权和召回数量不足。

> **面试口述**
> MMR 解决什么问题？lambda 怎么权衡？

追问

Q：MMR 的 lambda 怎么调？A：问答偏相关（0.7–0.9），探索型任务可降低；用评测看答案质量和重复率。

对应任务：学习舱 `w05d3`（4 个）

☐ DAY 4 上下文拼装、引用与拒答

引用不是装饰，是让答案可验证、让错误可定位的基础设施。

核心知识

- 上下文拼装：按相关性排序、控制 token 预算、保留来源/页码、避免重复。
- 引用粒度越细越好：chunk id + 原文片段 + 页码/段落。
- 生成后校验引用 id 存在、片段确实在上下文中、答案能被片段支持。
- 无证据时拒答；拒答阈值要在评测集上调，权衡误拒和漏拒。
- 拒答也要给下一步建议（缩小范围、补充文档、转人工）。

再深入

- 机制：context assembly 把候选 chunk 排序、去重、裁剪到 token 预算；citation 把答案片段映射回来源。
- 边界：引用 id 必须存在且真的支持结论；无证据应拒答；上下文顺序会影响模型注意力。

常见坑

- 让模型“自由发挥”，再补引用；引用和答案对不上。
- 拒答太激进，用户问什么都说不知道；要用评测调阈值。

> **面试口述**
> 怎么让引用可信，而不是装饰？

追问

Q：怎么判断引用是否支持答案？A：规则校验 id + 原文包含，再用 NLI/LLM 判断蕴含，最后人工抽检。

对应任务：学习舱 `w05d4`（4 个）

☐ DAY 5 最小 RAG 评测

没有评测集，你无法知道是检索坏了还是生成坏了。

核心知识

- 评测集：question、reference answer、relevant chunk ids、是否可回答。
- 检索指标：Recall@K、MRR、命中率；生成指标： faithfulness、答案相关性、引用命中。
- 失败分类：切分、召回、排序、上下文、生成、引用、权限。
- 人工抽检不可省；自动指标有偏差。
- 评测集要版本化，改动后跑回归。

再深入

- 机制：评测集把“我觉得好”变成可比较的数字；检索指标和生成指标要分开看。
- 边界：Recall@K 高不代表答案好；faithfulness 高也可能是“只回答安全的部分”；评测集会被调参污染。

常见坑

- 只用模型生成的题，题目和文档分布不一致。
- 只看一个总分，不定位失败类型，无法改。

> **面试口述**
> Recall@K 和 faithfulness 分别衡量什么？一个高另一个低说明什么？

追问

Q：怎么避免自己出题自己满分？A：题源来自真实用户/文档 FAQ/专家，保留 held-out，人工抽检并做跨版本对比。

对应任务：学习舱 `w05d5`（4 个）

☐ DAY 6 Lab：mini-rag 原型

先跑通闭环，再逐步替换每个环节。

核心知识

- 最小架构：ingest → chunk → embed → store → retrieve → prompt → answer → cite。
- 先做 CLI/单文件，再做 API；先 SQLite/pgvector，再考虑专用向量库。
- 增量索引用内容 hash + 唯一约束/upsert，失败任务可重试。
- 评测脚本独立于 UI，能命令行复现。
- README 写清架构、启动、评测结果、已知问题。

再深入

- 机制：最小 RAG 是 ingest → chunk → embed → store → retrieve → prompt → answer → cite 的闭环。
- 边界：先跑通再优化；没有 chunk 原文和 metadata，引用/排障都做不了；没有评测无法比较。

常见坑

- 一开始就上复杂框架，出问题不知道是哪一层。
- 不保存 chunk 原文和 metadata，引用/排查都做不了。

> **面试口述**
> 你的 RAG baseline 架构是什么？三个瓶颈在哪？

追问

Q：第一个瓶颈通常在哪？A：切分和召回最常见；先用失败样本定位，不要一上来换模型。

对应任务：学习舱 `w05d6`（4 个）

□ DAY 7 周复盘：RAG 基础

RAG 的每个环节都要有指标，否则优化就是碰运气。

核心知识

- 能画出完整链路并标注失败点。
- 能解释 chunk size/overlap 的取舍。
- 能说清 metadata filter 为什么必须在检索层。
- 能用 Recall@K 和 faithfulness 区分检索/生成问题。

再深入

- 机制：RAG 的失败可以分层：解析/切分、embedding、召回、排序、上下文、生成、引用、权限。
- 边界：不同层的症状可能相似（答错），必须用 trace 和评测区分；没有 baseline 无法归因。

常见坑

- 只关注“答得漂亮”，忽略“引用是否正确”。
- 评测集只在最后做，前期改动没有基线。

> **面试口述**
>
> 用 3 分钟讲 RAG 全链路和失败模式。

追问

Q：Recall@K 高但答案差，可能是什么？A：上下文噪声、排序靠后、模型没用证据、prompt 约束不足或引用错位。

对应任务：学习舱 `w05d7`（4 个）

W06

高级检索与 RAG 评测

7 节知识卡 · 先回忆，再对照

☐ DAY 1 BM25 与混合检索

向量检索擅长语义，BM25 擅长精确词；两者互补比二选一更稳。

核心知识

- BM25 基于词频、逆文档频率和文档长度归一化，对专有名词、编号、错误码很有效。
- 中文 BM25 需要分词；英文注意 stemming/停用词。
- 混合检索：dense + sparse 各自召回，再融合。
- RRF 按排名融合：score = Σ 1/(k + rank)，不需要分数归一化，稳健。
- 加权分数融合需要归一化和调权重，适合有标注数据时。

再深入

- 机制：BM25 用词频、IDF 和文档长度归一化打分；RRF 用排名融合多路召回，避免分数归一化。
- 边界：中文不分词 BM25 效果差；RRF 丢失分数强弱；混合检索增加索引和延迟。

常见坑

- 直接相加 BM25 分数和余弦分数，量纲不同，结果被某一路主导。
- 中文不分词就上 BM25，召回会很差。

> **面试口述**
> BM25 为什么能补向量检索？RRF 相比加权融合有什么优缺点？

追问

Q：RRF 的 k 怎么选？A：常用 60；k 越大排名差异越平滑，需用评测集调。

对应任务：学习舱 `w06d1`（4 个）

☐ DAY 2 Rerank：用精度换延迟

先便宜地召回，再昂贵地精排。

核心知识

- bi-encoder 分别编码 query/doc，可预计算，适合召回；cross-encoder 联合编码，适合精排。
- rerank 放在召回之后：top 50–100 精排到 top 5–10。
- 候选越多质量可能越好，但延迟/成本线性增加。
- 常用模型：bge-reranker、Cohere Rerank；本地部署要注意显存和批处理。
- 评测要同时看 Recall@5、答案质量、P95 延迟。

再深入

- 机制：bi-encoder 分别编码 query/doc，可预计算；cross-encoder 联合编码，能建模细粒度交互。
- 边界：rerank 只适合少量候选；候选越多延迟越高；模型领域不匹配会误排。

常见坑

- 对全库做 cross-encoder，延迟不可接受。
- 只加 rerank 不建评测集，感觉变好但指标没升。

> **面试口述**
> bi-encoder 和 cross-encoder 的区别？rerank 的延迟怎么控制？

追问

Q：rerank 提升不明显怎么办？A：先看召回是否已包含正确文档；没有召回，rerank 无从发挥。

对应任务：学习舱 `w06d2`（4 个）

☐ DAY 3 Query 改写：多查询、HyDE 与分解

用户的问题往往不是最好的检索 query。

核心知识

- query rewrite：补全指代、去掉口语噪声、提取关键词。
- multi-query：生成多个查询，分别召回后去重融合。
- HyDE：先生成假设答案，再用假设答案检索；适合短 query，但有幻觉偏移风险。
- 问题分解：多跳问题拆成子问题，分别检索再汇总。
- 改写会增加模型调用、延迟和成本；先看失败 query 类型再决定。

再深入

- 机制：query 改写补全指代、扩展同义词、分解多跳问题；HyDE 用假设答案做检索。
- 边界：改写可能改变意图；HyDE 的假设答案可能幻觉；多查询增加成本和延迟。

常见坑

- 所有 query 都改写，简单问题被改复杂。
- HyDE 的假设答案编造了错误事实，检索被带偏。

> **面试口述**
> HyDE 的原理和风险？什么场景适合用？

追问

Q：什么时候不该用 query 改写？A：query 已经明确、延迟敏感、改写模型不可靠或成本高于收益时。

对应任务：学习舱 w06d3（4 个）

☐ DAY 4 Parent-child、上下文压缩与引用对齐

小块负责“找得准”，大块负责“读得懂”。

核心知识

- parent-child：子块检索，命中后返回父块；兼顾精度和上下文。
- sentence window：命中句子，扩展前后句；适合问答。
- 上下文压缩：摘要/抽取/去重，降低 token 和噪声，但可能压掉证据。
- 引用对齐：chunk id → 原文位置；尽量细到句子/页码。
- 存储要保留父子映射和原文，引用才能回溯。

再深入

- 机制：parent-child 用小块检索、父块生成；上下文压缩减少 token 和噪声。
- 边界：父块太大会引入无关内容；压缩可能删掉关键证据；引用粒度会变粗。

常见坑

- 父块太大，把无关内容也塞给模型，成本上升。
- 压缩后引用指向的原文不在了，无法验证。

> **面试口述**
> parent-child 切分解决什么问题？代价是什么？

追问

Q：parent-child 的存储怎么设计？A：chunk 表存 parent_id，检索子块后 join 父块；原文位置单独存。

对应任务：学习舱 w06d4（4 个）

☐ DAY 5 RAGAS 与评测体系

自动指标用来回归和筛坏例，最终判断仍要人工和业务指标。

核心知识

- faithfulness：答案是否忠于上下文。
- answer relevancy：答案是否切题。
- context precision：相关上下文是否排在前面。
- context recall：回答所需证据是否被召回。
- judge 有偏差，需人工校准；指标要跟业务目标绑定。

再深入

- 机制：RAGAS 用 LLM judge 计算 faithfulness、answer relevancy、context precision/recall。
- 边界：judge 有偏差、成本高、对 prompt 敏感；指标之间可能冲突。

常见坑

- 把 RAGAS 分数当唯一真相，忽略人工抽检。
- 调参反复看同一评测集，造成过拟合。

> **面试口述**
> RAGAS 指标能替代人工评测吗？为什么？

追问

Q：RAGAS 分数下降但人工觉得更好，怎么办？A：检查 judge 偏差和评测集分布，不能只看自动分数。

对应任务：学习舱 w06d5（4 个）

☐ DAY 6 Lab：优化版 RAG 与消融

一次只改一个变量，才能知道是谁带来的提升。

核心知识

- 消融维度：切分、embedding、hybrid、rerank、rewrite、parent-child。
- 记录质量、延迟、成本三组指标，选满足 SLO 的组合。
- 默认配置不要堆满所有优化；保留开关，按场景切换。
- 失败案例比平均分更能指导下一步。

再深入

- 机制：消融实验一次只改一个变量，固定数据、模型、prompt 和评测集。
- 边界：多变量同改无法归因；评测集太小波动大；离线指标不等于线上体验。

常见坑

- 一次改三个变量，无法归因。
- 只看离线指标，不看线上延迟和成本。

> **面试口述**
> 你怎么证明一次 RAG 优化真的有效？

追问

Q：怎么选默认配置？A：在满足 SLO 的前提下，选质量/延迟/成本综合最优，并保留开关。

对应任务：学习舱 w06d6（4 个）

☐ DAY 7 周复盘：高级检索

优化的终点不是“指标最高”，而是“在 SLO 下最合适”。

核心知识

- 能解释 hybrid、RRF、rerank、rewrite 的适用场景和代价。
- 能设计一组消融实验并解释结果。
- 能说清 RAGAS 四个指标的含义和局限。
- 能根据失败类型选择优化手段，而不是盲目加组件。

再深入

- 机制：高级检索是“召回 + 排序 + 改写 + 上下文”的组合优化，不是组件堆叠。
- 边界：每加一个组件都增加延迟、成本和维护复杂度；没有评测就无法证明收益。

常见坑

- 优化组件越加越多，系统复杂到无法维护。
- 没有 baseline，面试时说不清“提升了多少”。

> **面试口述**
>
> 用 3 分钟讲 RAG 优化的取舍。

追问

Q：为什么不全开？A：可能超过延迟预算、增加故障面，且部分场景没有收益。

对应任务：学习舱 `w06d7`（4 个）

W07

项目一（上）：企业知识库 RAG

7 节知识卡 · 先回忆，再对照

☐ DAY 1 需求、架构与验收指标

项目一开始就要知道“什么算成功”，否则做完只是 demo。

核心知识

- 需求：谁用、问什么、文档从哪来、答错/答不出怎么处理。
- SLO：召回、忠实度、P95 首 token、可用性、单次成本。
- 数据模型：tenant、doc、chunk、conversation、message、task。
- 威胁模型：越权、注入、数据泄露、恶意文件、成本攻击。
- 架构：接入层、ingestion、检索、生成、评测、观测、运维。

再深入

- 机制：PRD 定义用户/场景/边界，SLO 定义可接受的质量和延迟，架构图定义组件和数据流。
- 边界：只定功能不定指标，做完无法验收；过度设计会拖慢主链路。

常见坑

- 先选技术栈再想需求，最后发现向量库/模型不匹配。
- 没有基线，做完不知道有没有变好。

> **面试口述**
> 为什么项目一开始就要定评测集和 SLO?

追问

Q：SLO 怎么定？A：参考业务容忍度、baseline 和成本；先定可测指标，再逐周提升。

对应任务：学习舱 `w07d1`（3 个）

☐ DAY 2 Ingestion：上传、解析、切分、入库

文档处理是长耗时、易失败的流水线，必须异步、可重试、可观测。

核心知识

- 上传：大小/类型限制、病毒扫描、租户校验、存储路径隔离。
- 解析：PDF/Word/Markdown，保留标题、页码、表格；失败要记录原因。
- 切分：按结构 + token 上限 + overlap；写入 metadata。
- 去重：内容 hash + 唯一约束；更新用版本号/upsert。
- 任务表：status、error、retry_count、doc_id；失败可重入，死信可人工处理。

再深入

- 机制：ingestion 是异步流水线：上传、解析、清洗、切分、embedding、入库；每步都可能失败。
- 边界：同步解析大文件会超时；重复上传会重复索引；坏文件会阻塞队列。

常见坑

- 同步解析大文件，HTTP 超时，用户以为失败又重复上传。
- 只追加新 chunk 不删旧版本，召回过期内容。

> **面试口述**
> ingestion 为什么要异步？怎么保证可重试和幂等?

追问

Q：怎么保证幂等？A：文档 id + 内容 hash + 唯一约束/upsert；重复任务先查状态。

对应任务：学习舱 `w07d2`（2 个）

☐ DAY 3 索引与检索

索引是“可重建的派生数据”，要能增量更新和一致性校验。

核心知识

- embedding 批处理 + 限流 + 失败重试；记录模型版本和维度。
- pgvector 建 HNSW；写入时带 tenant/doc/version metadata。
- 检索：dense + BM25 + filter；先过滤再召回，避免越权和数量不足。
- 增量索引：文档 id + 内容 hash + 版本；旧 chunk 删除/标记失效。
- 一致性校验：文档数、chunk 数、索引数对账；失败任务重跑。

再深入

- 机制：索引是可重建的派生数据；增量更新要处理旧 chunk 失效、版本和一致性。
- 边界：只追加不删会召回过期内容；向量与元数据不同步会出现“有向量没原文”。

常见坑

- 向量库和元数据分两套写，事务不一致，出现“有向量没原文”。
- 更新文档只改内容不换版本，缓存和引用指向旧内容。

> **面试口述**
> 增量索引怎么避免重复和漏索引?

追问

Q：怎么对账？A：文档数、chunk 数、索引数、失败任务数定期比对，异常告警。

对应任务：学习舱 `w07d3`（2 个）

☐ DAY 4 问答、引用与拒答

先保证“有据可查”，再追求“答得漂亮”。

核心知识

- 上下文拼装：按相关性排序、去重、控制 token、保留来源。
- 引用：chunk id + 原文片段 + 页码/段落；生成后校验。
- 拒答：检索分数 + 模型判断 + 无有效引用；阈值用评测集调。
- 流式回答：先返回检索状态/引用，再流式输出正文，体验更好。
- 会话历史只带最近相关轮次，避免上下文污染。

再深入

- 机制：回答链路是检索→排序→拼上下文→生成→引用校验；拒答是产品能力，不是异常。
- 边界：引用必须可验证；无答案时不要编；拒答太激进会伤体验。

常见坑

- 引用只显示文档名，无法定位；用户无法验证。
- 把“不知道”写成“根据资料……”然后编造，仍算幻觉。

> **面试口述**
> 拒答策略怎么设计？误拒和漏拒怎么权衡?

追问

Q：怎么调拒答阈值？A：在评测集上看误拒/漏拒曲线，按业务容忍度选点。

对应任务：学习舱 `w07d4`（2 个）

☐ DAY 5 多租户与权限

最危险的实现是“先全库召回，再在应用层过滤”。

核心知识

- tenant_id 进入所有表、chunk metadata、检索条件、缓存 key、日志。
- 数据库行级安全（RLS）可作为第二道防线；应用层仍要校验。
- API 鉴权：JWT/API key → 租户/用户 → 权限；不要只信前端。
- 缓存 key：tenant + doc version + query hash；否则串答案。
- 审计：谁在何时问了什么、命中了哪些文档、返回了什么。

再深入

- 机制：多租户隔离要贯穿数据、检索、缓存、日志和审计；tenant_id 是强制过滤条件。
- 边界：应用层过滤太晚；缓存 key 不带租户会串答案；RLS 不是万能，应用层仍要校验。

常见坑

- 只在应用层过滤，top-k 先被其他租户内容占满，召回不足且越权。
- 缓存 key 不带租户/版本，A 租户看到 B 租户答案。

> **面试口述**
> 多租户向量检索怎么防止越权？为什么要“检索前过滤”？

追问

Q：越权测试怎么做？A：构造 A 租户查 B 文档、猜 id、共享缓存、跨租户 embedding 等用例，纳入回归。

对应任务：学习舱 `w07d5`（2 个）

☐ DAY 6 集成、端到端测试与 demo

演示不是展示功能列表，而是讲清一个真实问题如何被解决。

核心知识

- 最小 UI/CLI：上传、索引状态、提问、引用、拒答。
- 端到端测试：上传 → 解析 → 索引 → 问答 → 引用完整链路。
- 错误状态要可见：解析失败、索引中、无权限、上游超时。
- demo 脚本：问题 → 约束 → 架构 → 现场操作 → 指标 → 失败案例。
- 录屏 3 分钟，先讲价值再讲技术栈。

再深入

- 机制：demo 是价值证明：问题→约束→架构→操作→指标→失败；不是功能列表。
- 边界：网络/模型可能超时，准备离线兜底；只演成功路径会被追问卡住。

常见坑

- 演示时网络/模型超时，没有离线兜底或预置结果。
- 只演示成功路径，面试官一问边界就卡住。

> **面试口述**
> 演示项目时怎么讲架构？先讲什么、后讲什么？

追问

Q：演示最容易被问什么？A：切分、召回、引用、权限、成本和失败案例。

对应任务：学习舱 `w07d6`（4 个）

☐ DAY 7 周复盘：项目一主链路

主链路稳、指标真、权限不漏，比功能多更重要。

核心知识

- 对照验收指标自测：召回、忠实度、引用、延迟、成本。
- 列出最大风险：权限、性能、评测、成本、体验。
- 项目复盘：完成了什么、没完成什么、下周优先级。
- 准备生产化清单：容器、缓存、监控、压测、文档。

再深入

- 机制：项目复盘按“目标—结果—差距—原因—下一步”闭环，不是写流水账。
- 边界：主链路不稳就加功能会扩大故障面；没有指标就无法判断优先级。

常见坑

- 主链路还有硬编码/手工步骤，却开始加新功能。
- 没有真实文档样本，评测和演示都不可信。

> **面试口述**
> 项目一当前最大的风险是什么？你准备怎么修？

追问

Q：下周先修什么？A：先修会导致数据泄露或无法上线的，再修体验和成本。

对应任务：学习舱 `w07d7`（4 个）

W08

项目一（下）：生产化与交付

7 节知识卡 · 先回忆，再对照

☐ DAY 1 Docker Compose 与配置管理

新机器 10 分钟内能跑起来，才算可交付。

核心知识

- Compose 编排 api、postgres、redis、可选 langfuse；服务名即 DNS。
- 数据卷持久化数据库；healthcheck 判断就绪；depends_on 配 condition。
- 配置走环境变量；.env.example 提供模板，绝不提交真实密钥。
- 启动时跑 migration；失败要快速退出，不要带病启动。
- 只暴露必要端口；生产用反向代理和 TLS。

再深入

- 机制：Compose 用服务名做 DNS，卷持久化数据，healthcheck 让依赖方等待就绪。
- 边界：depends_on 不等于健康；migration 失败要快速退出；密钥不能进镜像。

常见坑

- 容器启动顺序靠 sleep，数据库没就绪就迁移失败。
- 镜像里打包 .env，密钥泄露。

> **面试口述**
> Compose 的网络、卷和健康检查怎么设计？

追问

Q：启动顺序怎么保证？A：healthcheck + depends_on condition；应用内重试连接，不要只靠 sleep。

对应任务：学习舱 `w08d1`（2 个）

☐ DAY 2 缓存、限流与成本控制

缓存是降本利器，但一致性、租户隔离和过期策略必须一起设计。

核心知识

- 精确缓存 key：tenant + doc version + query hash + prompt version + model。
- 语义缓存按相似度命中，命中率高但有串答案风险；要设阈值、TTL、可关闭。
- 限流按用户/租户/模型维度；保护上游配额和成本。
- token 成本统计：输入/输出分开，检索/rerank 也算。
- 预算告警：日/月限额，异常流量自动降级。

再深入

- 机制：精确缓存按 key 命中，语义缓存按向量相似度命中；缓存降低延迟和成本。
- 边界：语义缓存可能串答案；文档更新/权限变化会让缓存过期；缓存 key 必须带租户和版本。

常见坑

- 缓存 key 不带版本，文档更新后仍返回旧答案。
- 语义缓存阈值太低，把不同意图的问题混在一起。

> **面试口述**
> 语义缓存的风险是什么？怎么防止串答案？

追问

Q：缓存怎么失效？A：文档版本变化、权限变化、prompt/模型变化时失效；设 TTL 兜底。

对应任务：学习舱 `w08d2`（2 个）

☐ DAY 3 可观测性：trace、token、延迟、错误

没有 trace，线上问题只能靠猜。

核心知识

- 一次请求记录：request id、tenant、query、改写、召回 chunk/score、rerank、prompt 版本、模型、token、首 token/总延迟、引用、拒答原因。
- trace 要能回放：输入、输出、耗时、错误、成本。
- 指标：P50/P95、错误率、缓存命中率、token/请求、单次成本。
- 日志分级：INFO 业务事件、WARN 可恢复异常、ERROR 需处理。
- 告警：错误率、P95、成本、上游限流。

再深入

- 机制：trace 记录一次请求的完整调用链；指标聚合延迟、错误、token 和成本。
- 边界：只记录最终答案无法定位问题；日志含隐私会合规风险；采样率太高成本高。

常见坑

- 只记录最终答案，不记录检索和 prompt；无法定位是召回还是生成问题。
- 日志里打印完整用户隐私/文档内容，造成合规风险。

> **面试口述**
> 一次 RAG 请求要记录哪些字段？为什么？

追问

Q：线上问题怎么复现？A：用 trace 的输入、检索结果、prompt 和模型参数重放。

对应任务：学习舱 `w08d3`（2 个）

☐ DAY 4 评测回归与 CI

评测要像单测一样，每次改动都能跑、能比较、能阻止回归。

核心知识

- 固定评测集 + 版本；每次 PR 跑检索/生成指标。
- 设阈值和允许波动；LLM judge 成本高时抽样。
- 失败样本人工复核；结果存档，支持版本对比。
- 评测要覆盖：正常、边界、无答案、越权、注入。
- 报告要能回答：变好了吗？哪类问题变差？成本/延迟多少？

再深入

- 机制：评测 CI 在每次 PR 跑固定集，阈值阻止回归；结果存档支持版本对比。
- 边界：judge 成本高要抽样；评测集太小波动大；阈值太严会阻塞正常迭代。

常见坑

- 评测集和训练/调参数据混在一起，分数虚高。
- 只看平均分，忽略某一类问题大幅退化。

> **面试口述**
> 怎么把 RAG 评测接入 CI？阈值怎么定？

追问

Q：阈值怎么定？A：用历史波动和业务容忍度定，允许小波动，关键指标单独告警。

对应任务：学习舱 `w08d4`（2 个）

☐ DAY 5 压测与性能优化

先定位瓶颈，再优化；不要凭感觉换模型。

核心知识

- 分段计时：鉴权、检索、rerank、prompt、模型、DB、序列化。
- 压测阶梯增加并发，观察吞吐/延迟拐点。
- 常见瓶颈：上游配额、连接池、DB、rerank CPU、上下文过大。
- 优化手段：缓存、批处理、连接复用、并发限流、上下文裁剪。
- 报告写清环境、并发、数据量、P95、错误率和优化前后对比。

再深入

- 机制：压测找容量边界和瓶颈；分段计时区分检索、rerank、模型、DB 和序列化。
- 边界：只压 healthcheck 没意义；压测环境与生产差异大；长尾请求会拉高 P95。

常见坑

- 只压 healthcheck，不压真实链路。
- 压测环境和生产差距太大，数据不可参考。

> **面试口述**
> P95 很高，你怎么定位是检索、rerank 还是模型慢？

追问

Q：P95 高先看什么？A：trace 长尾、连接池、上游限流、DB 慢查询和上下文大小。

对应任务：学习舱 `w08d5`（2 个）

☐ DAY 6 交付：README、架构图、视频、文章

面试官先看 README，再决定要不要看代码。

核心知识

- README：问题、架构、启动、指标、取舍、失败复盘、限制。
- 架构图：接入、ingestion、检索、生成、评测、观测。
- 3 分钟视频：问题 → 演示 → 指标 → 取舍。
- 技术文章：背景 → 方案 → 数据 → 失败 → 结论。
- 仓库清理：无密钥、无大文件、commit 清晰、tag v1.0。

再深入

- 机制：README 是项目入口，架构图是认知地图，指标是可信度，失败复盘是深度。
- 边界：只有安装命令无法证明能力；指标没有口径会被质疑；视频太长没人看。

常见坑

- README 只有安装命令，没有指标和取舍；无法证明能力。
- 视频只是录屏不说话，观众不知道在解决什么。

> **面试口述**
> 用 10 分钟讲项目一，你会怎么组织？

追问

Q：面试官最想看什么？A：你解决了什么问题、怎么验证、失败过什么、如果重做怎么改。

对应任务：学习舱 `w08d6`（4 个）

☐ DAY 7 周复盘 + 项目一模拟面试

讲不清的地方，就是还没真懂。

核心知识

- 模拟追问：为什么这样切分？为什么 hybrid？rerank 延迟？多租户？评测怎么建？
- 每个技术选择准备替代方案和代价。
- 记录被问住的 5 个点，补数据/补实验。
- 更新简历 bullet，确保每个数字有来源。

再深入

- 机制：模拟面试检验结构、取舍和抗压；录音回听比自我感觉可靠。
- 边界：只背答案会被追问击穿；只讲成功不讲失败显得不真实。

常见坑

- 只准备“成功故事”，被问失败就慌。
- 指标说不清口径，面试官会怀疑真实性。

> **面试口述**
> 项目一最难的三个决策是什么？替代方案是什么？

追问

Q：被问住怎么办？A：承认边界，拆解问题，给假设和验证方式，不要编数据。

对应任务：学习舱 `w08d7`（3 个）

W09 Agent 基础与工具调用

7 节知识卡 · 先回忆，再对照

☐ DAY 1 Agent 的适用边界

能不用 Agent 就不用；路径不确定、需要工具组合时再上。

核心知识

- workflow：控制流由代码预先定义，可预测、易测试、易审计。
- agent：由模型决定下一步，灵活但非确定性、成本高、难调试。
- 适合 Agent：开放式研究、多工具组合、路径不确定、需要根据中间结果调整。
- 不适合：步骤固定、合规高风险、强可解释、成本/延迟敏感、副作用大且难回滚。
- 设计顺序：先 workflow，再单 Agent，最后才多 Agent。

再深入

- 机制：workflow 的控制流由代码定义，agent 由模型动态决定下一步；前者可预测，后者灵活。
- 边界：合规高风险、步骤固定、成本敏感时优先 workflow；agent 增加不确定性和调试成本。

常见坑

- 为了“看起来高级”上 Agent，结果比 workflow 更慢更贵更不稳。
- 没有成功标准和任务集，无法判断 Agent 是否有用。

> **面试口述**
> 什么时候不该用 Agent？你会怎么判断？

追问

Q：怎么判断该不该用 agent？A：先问路径是否确定、是否需要动态工具、失败代价是否可接受。

对应任务：学习舱 `w09d1`（4 个）

☐ DAY 2 手写 ReAct 循环

ReAct 是“思考 → 行动 → 观察”的循环，不是让模型自由发挥。

核心知识

- 每轮：模型输出 thought/action，应用执行工具，把 observation 回填。
- 需要停止条件：完成信号、最大步数、预算耗尽、连续失败。
- 解析动作要健壮：JSON/函数调用优先，避免正则解析自由文本。
- 观察结果要截断/结构化，避免把超长网页塞回上下文。
- 全链路 trace：每步输入、工具、参数、结果、耗时、token。

再深入

- 机制：ReAct 循环是 thought → action → observation；应用执行工具并把结果回填。
- 边界：没有最大步数会死循环；工具错误直接抛异常会中断；观察结果可能被注入。

常见坑

- 没有最大步数，模型可以无限循环。
- 工具报错直接抛异常，Agent 直接崩；应返回结构化错误让模型决策。

> **面试口述**
> ReAct 循环的核心风险有哪些？你怎么止损？

追问

Q：怎么检测死循环？A：工具+参数哈希重复、步数上限、状态无变化、连续相同 observation。

对应任务：学习舱 `w09d2`（4 个）

☐ DAY 3 工具设计、参数校验与幂等

工具是 Agent 的手，手要有权限、有边界、有痛觉。

核心知识

- 工具名动词化，描述写清“何时用/何时不用”，参数用 JSON Schema。
- 服务端校验参数：类型、枚举、范围、白名单；不要只信模型输出。
- 错误结构化：错误类型、原因、是否可重试、建议动作。
- 有副作用的工具要幂等：幂等 key、去重表、事务。
- 高风险工具加审批、超时、沙箱、审计。

再深入

- 机制：工具是带 schema 的函数；模型只生成调用意图，应用负责校验、权限、执行和错误处理。
- 边界：描述不清会选错工具；参数不校验会注入；有副作用的工具重试要幂等。

常见坑

- SQL 工具给读写权限，模型一句 DROP 就出事。
- 重试“发通知”没有幂等，用户收到多条。

> **面试口述**
> 怎么防止工具参数注入？权限应该放在哪一层？

追问

Q：工具描述怎么写？A：动词化名称、何时用/不用、参数类型/枚举/示例、错误和权限边界。

对应任务：学习舱 `w09d3`（4 个）

☐ DAY 4 Memory：短期、长期与遗忘

记忆不是越多越好，错误记忆比没有记忆更糟。

核心知识

- 短期记忆：当前会话消息，按 token 预算裁剪。
- 长期记忆：用户偏好、稳定事实、任务结果；要有来源、置信度、时间。
- 写入策略：只写明确、稳定、可验证的信息；不要每轮都写。
- 检索策略：按当前任务检索相关记忆，避免全量注入。
- 遗忘/删除：过期、纠错、用户撤回；敏感信息不写或加密。

再深入

- 机制：短期记忆是会话上下文，长期记忆是跨会话的偏好/事实；都需要写入、检索、更新和删除策略。
- 边界：每轮都写会污染；模型推断不是事实；敏感信息不能随便存。

常见坑

- 把模型推断写入长期记忆，后续一直被错误前提带偏。
- 没有删除入口，用户无法纠正隐私信息。

> **面试口述**
> 记忆什么时候写？写错了怎么办？

追问

Q：记忆冲突怎么办？A：以最新明确指令为准，保留来源和时间，允许用户纠正/删除。

对应任务：学习舱 w09d4（4 个）

☐ DAY 5 Guardrails、预算与红队

Agent 一定要有刹车，尤其是能花钱、能写数据的工具。

核心知识

- 预算：最大步数、最大 token/金额、最大时间；超限降级。
- 工具权限：白名单、最小权限、审批、沙箱。
- 输入/输出护栏：敏感信息、注入、格式、越权。
- 重复动作检测：相同工具+参数哈希，避免死循环。
- Kill switch：异常时立即停止，保留现场 trace。

再深入

- 机制：护栏是纵深防御：输入校验、工具权限、预算、审批、输出过滤和 kill switch。
- 边界：只限步数不限 token 仍会烧钱；审批只有“通过”没有拒绝等于没审批。

常见坑

- 只限步数不限 token，单步超长上下文仍能烧钱。
- 审批只做“通过”，没有拒绝/修改/超时分支。

> **面试口述**
> Agent 的预算怎么设？超预算后怎么降级？

追问

Q：预算怎么设？A：按任务价值设最大步数/token/金额/时间，超限给当前结论和待办。

对应任务：学习舱 w09d5（4 个）

☐ DAY 6 Lab：mini-agent 完整版

先手写，再用框架；否则你不知道框架帮你做了什么。

核心知识

- 组件：模型适配器、工具注册表、循环、状态、记忆、trace、护栏。
- 工具注册表统一 schema、执行、错误、超时、权限。
- 状态要可序列化，方便恢复和回放。
- 评测：任务成功率、平均步数、工具错误率、人工干预率。
- 保留手写版，面试时能讲底层。

再深入

- 机制：mini-agent 包含模型适配、工具注册、循环、状态、记忆、trace 和护栏。
- 边界：全塞进 prompt 无法测试；没有 trace 无法复盘；没有任务集无法证明成功率。

常见坑

- 把所有逻辑塞进一个 prompt，没有可测试的代码结构。
- 没有 trace，失败后无法复盘。

> **面试口述**
> 手写 Agent 和 LangGraph 的区别？什么时候用框架？

追问

Q：手写和框架怎么选？A：简单任务手写更可控；复杂状态/审批/恢复用 LangGraph。

对应任务：学习舱 w09d6（4 个）

☐ DAY 7 周复盘：Agent 基础

Agent 的难点不是“让它动”，是“让它停”和“让它可控”。

核心知识

- 能判断 workflow vs agent。
- 能画 ReAct 循环并标出失败点。
- 能设计工具 schema、错误协议、幂等和权限。
- 能说清记忆写入/删除策略和预算控制。

再深入

- 机制：Agent 的难点是停止条件、工具边界、权限和可观测性，不是“让模型自由发挥”。
- 边界：没有成功标准就无法评测；多智能体不是默认答案。

常见坑

- 只追求“多智能体”，单 Agent 还没跑稳。
- 没有任务集，无法证明成功率。

> **面试口述**
> 用 3 分钟讲 Agent 的组成和失败模式。

追问

Q：怎么证明 agent 可靠？A：固定任务集、多次运行分布、人工抽检和失败分类。

对应任务：学习舱 w09d7（4 个）

W10

LangGraph 与工作流

7 节知识卡 · 先回忆，再对照

☐ DAY 1 StateGraph：把 Agent 画成状态机

能用代码表达的控制流，不要交给模型即兴发挥。

核心知识

- StateGraph：节点（函数）处理状态，边决定下一步；条件边按状态分支。
- 状态 schema 要明确字段和 reducer；多个节点更新同一字段时的合并规则很关键。
- 节点职责单一：解析、检索、生成、校验、执行，各自可测试。
- 普通边是固定顺序；条件边用于分支、重试、循环、审批。
- 先把图画出，再写代码；图就是架构。

再深入

- 机制：StateGraph 用节点处理状态、边控制流转；条件边根据状态选择分支。
- 边界：状态字段没有 reducer 会被覆盖；节点职责过多难定位；图太复杂难维护。

常见坑

- 节点做太多事，失败无法定位。
- 状态字段没有 reducer，消息/候选被覆盖。

> **面试口述**
>
> LangGraph 的条件边解决什么问题？相比 while 循环的优势？

追问

Q：什么时候用子图？A：可复用的多步子流程，或需要独立状态/权限的模块。

对应任务：学习舱 `w10d1`（4 个）

☐ DAY 2 Checkpoint 与中断恢复

checkpoint 让工作流可以暂停、恢复、重放，而不是只能从头再来。

核心知识

- checkpointer 持久化每步状态；thread_id 隔离不同会话。
- 支持失败恢复：从最后一个成功节点继续。
- 支持人工审批：暂停后隔天恢复，不阻塞进程。
- 支持时间旅行：回到某步，修改状态重跑。
- 生产用 Postgres/Redis 持久化，注意并发、清理和隐私。

再深入

- 机制：checkpoint 持久化每步状态，thread_id 隔离会话；支持恢复、审批和时间旅行。
- 边界：thread_id 冲突会串状态；持久化敏感信息要加密/过期；并发写要处理冲突。

常见坑

- thread_id 冲突，两个用户共享状态。
- checkpoint 存了敏感信息，没有加密/过期策略。

> **面试口述**
>
> checkpoint 解决什么问题？thread_id 为什么关键？

追问

Q：checkpoint 存什么？A：状态、下一步、时间戳、版本；不要存无法序列化的对象。

对应任务：学习舱 `w10d2`（4 个）

☐ DAY 3 Human-in-the-loop：审批、修改、恢复

审批不是弹窗，是状态机里一个可恢复的节点。

核心知识

- 在关键动作前 interrupt，持久化待审批动作、参数、风险说明、发起人、过期时间。
- 审批结果：通过 / 拒绝 / 修改参数；都要写入状态。
- 恢复前重新校验参数、权限和时效；审批期间环境可能变化。
- 拒绝要走终止分支，不能只有“通过”。
- 审批记录进审计日志，支持追责。

再深入

- 机制：interrupt 在关键动作前暂停，状态持久化；resume 时重新校验并继续。
- 边界：审批只存内存会丢；恢复前不校验参数/权限会执行被篡改的动作；拒绝分支必须存在。

常见坑

- 审批只存在内存，服务重启后丢失。
- 恢复时直接执行，不校验参数是否被篡改。

> **面试口述**
> 人工审批节点怎么设计状态？恢复前为什么要重新校验？

追问

Q：审批状态包含什么？A：待执行工具、参数、风险、发起人、过期时间、决策和修改记录。

对应任务：学习舱 `w10d3`（4 个）

☐ DAY 4 多智能体：supervisor 与专家

多智能体不是高级版，是另一种权衡。

核心知识

- supervisor 负责分派任务、汇总结果；专家 Agent 负责具体领域。
- handoff 传递任务和上下文；共享状态要定义清楚。
- 多 Agent 增加通信成本、状态不一致、调试难度和延迟。
- 适合：任务可清晰分工、需要并行探索、不同工具权限。
- 不适合：简单强顺序任务；先用单 Agent + 好工具。

再深入

- 机制：supervisor 分派任务给专家 agent，汇总结果；共享状态和通信协议决定可靠性。
- 边界：多 agent 增加通信成本、状态不一致、延迟和调试难度；简单任务不需要。

常见坑

- 为多而多，最后成功率不如单 Agent。
- 共享状态没有版本/锁，专家之间互相覆盖。

> **面试口述**
> 多智能体什么时候是反模式？怎么判断是否值得拆？

追问

Q：什么时候多 agent 是反模式？A：任务强顺序、共享状态复杂、收益低于通信成本时。

对应任务：学习舱 `w10d4`（4 个）

☐ DAY 5 流式、可观测与任务评测

非确定性系统要看分布，不看单次成败。

核心知识

- 流式事件：节点开始/结束、工具调用、token、错误；前端可展示进度。
- 节点级 trace：输入、输出、耗时、token、错误。
- 任务评测：固定任务集 + 成功标准，多次运行看成功率、步数、成本、干预率。
- 失败分类：规划错、工具错、参数错、观察误读、循环、权限。
- 允许波动，报告分布和置信区间，不报单点。

再深入

- 机制：节点级 trace 记录输入/输出/耗时/token/错误；任务评测看成功率、步数、成本和干预率。
- 边界：单次成功不代表稳定；只看成功率不看成本会得出错误结论。

常见坑

- 只跑一次成功就宣布可用。
- 只看成功率，不看步数和成本。

> **面试口述**
> 怎么评测一个非确定性的 Agent？

追问

Q：非确定性系统怎么评测？A：固定任务集、多次运行、人工抽检、统计分布和置信区间。

对应任务：学习舱 `w10d5`（4 个）

☐ DAY 6 Lab：带审批的运营工作流

把状态图、checkpoint、审批、trace 串成一个能演示的闭环。

核心知识

- 流程：查数据 → 生成报告 → 审批 → 发送/执行 → 汇总。
- 审批支持通过/拒绝/改参/超时；拒绝走终止分支。
- 高风险动作前暂停；恢复前重新校验。
- trace 记录每步；失败可从 checkpoint 恢复。
- demo 要展示“暂停→改参→恢复”，这是差异化。

再深入

- 机制：运营工作流是“查数据→生成报告→审批→执行→汇总”的状态机。
- 边界：审批/执行耦合难测试；没有拒绝/超时分支不算可控；没有 trace 无法排障。

常见坑

- 演示只走“通过”，没有拒绝/修改，看不出可控性。
- 审批和执行业务逻辑耦合，无法单独测试。

> **面试口述**
> 用 2 分钟讲你的工作流状态机。

追问

Q：工作流怎么测试？A：节点单测 + 图级集成测试 + 失败恢复测试 + 审批分支测试。

对应任务：学习舱 `w10d6`（4 个）

☐ DAY 7 周复盘：LangGraph

显式状态 + 可恢复执行，是复杂 Agent 的生产化基础。

核心知识

- 能画状态图并解释每个节点/边。
- 能说清 checkpoint、thread_id、interrupt/resume。
- 能设计审批状态和拒绝分支。
- 能判断多智能体是否值得。

再深入

- 机制：显式状态 + 可恢复执行是复杂 agent 的生产化基础；while 循环适合简单任务。
- 边界：框架增加学习成本；状态设计不好会比 while 更难维护。

常见坑

- 只学 API，不理解状态和恢复，线上排障困难。
- 把审批当 UI 弹窗，服务重启就丢。

> **面试口述**
> LangGraph 相比 while 循环的优势？代价是什么？

追问

Q：LangGraph 的代价是什么？A：抽象和依赖增加，简单任务过度设计。

对应任务：学习舱 `w10d7`（4 个）

W11 项目二：Agent 工作流平台

7 节知识卡 · 先回忆，再对照

☐ DAY 1 需求、任务集与架构

先写任务集，再写 Agent；否则做完不知道好不好。

核心知识

- 任务集覆盖：简单、复杂多步、无解、越权、工具失败五类。
- 每条任务有明确成功标准：答案正确/动作完成/格式合法。
- 架构：API、编排引擎、工具运行时、状态存储、trace、评测。
- 权限矩阵：哪个工具、哪个角色、是否需要审批。
- 风险清单：越权、死循环、成本、记忆污染、工具失败。

再深入

- 机制：任务集定义成功标准和覆盖面，指导工具、状态和评测设计。
- 边界：全是顺风局无法覆盖真实失败；成功标准模糊无法自动判定。

常见坑

- 任务集全是“顺风局”，线上第一次遇到边界就崩。
- 成功标准模糊，无法自动判定。

> **面试口述**
> 为什么先写任务集再写 Agent?

追问

Q：任务集怎么扩充？A：从真实用户请求、失败日志、边界场景和红队用例中补。

对应任务：学习舱 `w11d1`（3 个）

☐ DAY 2 工具集：搜索、SQL、知识库、文件

工具是权限和风险的集中地，先设计边界再写实现。

核心知识

- 统一工具协议：名称、描述、参数 schema、超时、错误、权限。
- SQL 工具：只读账号、表白名单、参数化、行级权限、查询超时、行数限制。
- 知识库工具：复用项目一 RAG，返回引用。
- 文件工具：沙箱目录、扩展名白名单、大小限制。
- 通知/写操作：幂等 key、审批、审计。

再深入

- 机制：工具集统一 schema、错误、超时和权限；SQL/文件等高风险工具要最小权限。
- 边界：管理员权限会放大注入；没有超时会拖垮工作流；错误不结构化模型无法决策。

常见坑

- SQL 工具给管理员权限，一次注入全库泄露。
- 文件工具可以读任意路径，造成敏感文件泄露。

> **面试口述**
> 给 Agent 的 SQL 工具，最关键的安全措施有哪些?

追问

Q：工具超时怎么处理？A：返回结构化超时错误，Agent 决定重试/换工具/降级。

对应任务：学习舱 `w11d2`（2 个）

☐ DAY 3 状态机、审批与恢复

把高风险动作放进可暂停、可拒绝、可恢复的节点。

核心知识

- 图：理解 → 规划 → 工具 → 观察 → 生成 → 校验；高风险动作前审批。
- 审批状态：待审批动作、参数、风险、发起人、过期时间。
- 恢复前重新校验参数/权限/时效；拒绝走终止分支。
- 工具失败返回结构化错误，Agent 决定重试/换工具/终止。
- 重试要幂等、限次数、可观测。

再深入

- 机制：状态机把规划、工具、审批、执行分开；checkpoint 支持恢复。
- 边界：审批后不重新校验会执行过期/越权动作；工具重试不幂等会重复副作用。

常见坑

- 审批只做“通过”，没有拒绝和修改。
- 工具失败直接抛异常，整个工作流崩溃。

> **面试口述**
> 工具失败时 Agent 应该怎么处理？怎么避免无限重试?

追问

Q：怎么保证重试安全？A：幂等 key、去重表、事务和最大重试次数。

对应任务：学习舱 `w11d3`（2 个）

☐ DAY 4 记忆、权限与审计

Agent 可以记住该记的，但必须能删、能纠错、能审计。

核心知识

- 会话记忆：最近消息 + 摘要；长期记忆：稳定偏好/事实。
- 记忆条目带 source/confidence/expires_at；区分用户陈述与模型推断。
- 权限：工具级 + 数据级；执行前校验，不依赖模型自觉。
- 审计：谁、何时、对什么资源、执行什么动作、结果。
- 隐私：敏感信息不写记忆；支持删除和导出。

再深入

- 机制：记忆、权限、审计是 agent 落地的三条底线；模型不能自己决定权限。
- 边界：模型推断写入记忆会污染；审计只记最终答案无法还原；敏感信息不能明文存。

常见坑

- 把模型推断写入长期记忆，后续一直被错误前提带偏。
- 审计只记录最终答案，无法还原执行链。

> **面试口述**
> Agent 的记忆如何避免污染？权限和审计怎么设计?

追问

Q：记忆怎么删除？A：提供查看/删除/过期，删除时同步索引和缓存，并记录审计。

对应任务：学习舱 `w11d4`（2 个）

☐ DAY 5 Trace、评测与成本

每次运行都要能回放，才能定位“为什么失败、花了多少”。

核心知识

- trace：每步 thought/action/observation、工具参数、耗时、token、错误。
- 指标：成功率、平均步数、工具错误率、人工干预率、单任务成本。
- 成功率按任务成功标准判定，自动 + 人工抽检。
- 多次运行取分布，避免单次幸运成功。
- 成本按模型 + 工具 + 检索分别统计。

再深入

- 机制：trace 记录每次 tool call；评测看成功率、步数、成本、工具错误率和干预率。
- 边界：成功率定义太宽会虚高；只统计模型 token 会低估成本；单次运行不可靠。

常见坑

- 只统计模型 token，忽略检索/工具/重试成本。
- 成功率定义太宽，把“部分完成”算成功。

> **面试口述**
> Agent 的成功率怎么定义？为什么还要看步数和成本？

追问

Q：成功率和效率怎么平衡？A：先达质量门槛，再优化步数和成本；不能为了步数牺牲正确性。

对应任务：学习舱 `w11d5`（2 个）

☐ DAY 6 集成、demo 与文档

让面试官在 3 分钟内看到“可控”，而不是“多智能体”。

核心知识

- 最小 UI/CLI：发起任务、审批交互、运行历史、trace 查看。
- demo：正常任务 → 工具失败恢复 → 审批拒绝/改参。
- README：架构、工具协议、权限、评测、失败复盘。
- 项目二与项目一区分：重点是编排/审批/状态，不是重复 RAG。
- 视频 3 分钟：问题 → 流程 → 审批 → 指标。

再深入

- 机制：项目二证明“可控”：状态、工具协议、审批、恢复、trace 和评测。
- 边界：只演示成功路径看不出可控；重复项目一没有差异化。

常见坑

- 演示只走成功路径，看不出“可控”。
- 项目二和项目一讲成同一个东西，浪费一个项目。

> **面试口述**
> 用 10 分钟讲项目二，重点讲什么？

追问

Q：项目二和项目一的区别？A：项目一是检索与生成，项目二是编排、工具、审批和可观测。

对应任务：学习舱 `w11d6`（4 个）

☐ DAY 7 周复盘 + 项目二模拟面试

项目二的关键词是“可控”，不是“多智能体”。

核心知识

- 高频追问：为什么用 Agent？怎么防死循环/越权？怎么评测？
- 每个机制要有数据：成功率、步数、成本、干预率。
- 失败案例要具体：哪个工具、什么参数、怎么修。
- 与项目一的边界要清楚。

再深入

- 机制：项目二的高频追问围绕“为什么 agent、怎么防死循环/越权、怎么评测”。
- 边界：没有任务集和指标，回答会空；没有失败案例，显得不真实。

常见坑

- 讲了一堆框架名，讲不清状态和权限。
- 没有真实任务集，指标不可信。

> **面试口述**
> 项目二最容易被问倒的三个点是什么？

追问

Q：最容易被问倒什么？A：为什么不用 workflow、审批状态怎么恢复、成功率怎么定义。

对应任务：学习舱 `w11d7`（3 个）

W12

微调与数据工程

7 节知识卡 · 先回忆，再对照

☐ DAY 1 微调决策树

知识频繁变化用 RAG，风格/格式固定才考虑微调。

核心知识

- Prompt：先解决 80% 问题，成本最低、迭代最快。
- RAG：动态知识、需要引用/权限、可追溯。
- Fine-tune：固定风格/格式/领域语言、有高质量标注数据、需要小模型降本。
- 微调不适合注入频繁变化的事实；会过时且难追溯。
- 微调有维护成本：数据、训练、评测、版本、回滚。

再深入

- 机制：Prompt 解决表达，RAG 解决知识，微调解决风格/格式/领域模式；三者可叠加。
- 边界：频繁变化的事实不适合微调；数据不足/质量差时微调会放大问题；微调有版本和维护成本。

常见坑

- 一上来就微调，数据不够、评测没有，最后不如 Prompt。
- 用微调记事实，知识更新后要反复重训。

> **面试口述**
> 什么时候用 RAG，什么时候用微调？两者能叠加吗？

追问

Q：RAG 和微调能一起用吗？A：能；RAG 提供证据，微调固定风格/格式，注意评测和版本。

对应任务：学习舱 `w12d1`（4 个）

☐ DAY 2 数据工程：业务数据变训练样本

数据质量决定上限，训练技巧只能逼近上限。

核心知识

- 格式：instruction/input/output 或 chat messages；训练和推理模板要一致。
- chat template：角色标记、特殊 token、loss mask 必须匹配。
- 清洗：去重、去噪、长度控制、答案审核、隐私脱敏。
- 划分：train/validation/test 隔离，防止泄漏。
- 数据卡：来源、规模、字段、清洗、许可、隐私。

再深入

- 机制：训练数据要套 chat template、mask 掉非 assistant 部分；训练和推理模板必须一致。
- 边界：模板错会导致 loss 正常但效果差；数据泄漏会让评测虚高；隐私数据要脱敏。

常见坑

- 训练用一套模板，推理用另一套，loss 正常但效果差。
- 测试集和训练集重复，评测分数虚高。

> **面试口述**
> 训练数据的质量怎么保证？chat template 为什么重要？

追问

Q：怎么发现模板错？A：打印 token、对比推理 prompt、检查 loss mask 和特殊 token。

对应任务：学习舱 `w12d2`（4 个）

☐ DAY 3 LoRA/QLoRA 原理与超参

LoRA 冻结基座，只训练低秩适配器，用少量参数换领域能力。

核心知识

- LoRA：W' = W + BA，A/B 低秩；只训练 A/B，冻结原权重。
- rank 越大容量越强，但更易过拟合、显存更高；常用 8–64。
- alpha 常设为 rank 的 1–2 倍；dropout 防过拟合。
- QLoRA：4-bit 量化基座 + LoRA，进一步降显存，速度/质量需权衡。
- 目标模块通常选 attention 的 q/k/v/o，也可加 FFN。

再深入

- 机制：LoRA 冻结基座，训练低秩 A/B 矩阵；rank 控制容量，alpha 控制缩放。
- 边界：rank 越大越易过拟合、显存越高；目标模块选错效果差；adapter 保存/加载要匹配。

常见坑

- rank 盲目调大，数据少时直接过拟合。
- 只训练 adapter 但保存/加载方式不对，推理没生效。

> **面试口述**
> LoRA 为什么省显存？rank 怎么选？

追问

Q：QLoRA 和 LoRA 怎么选？A：显存不够用 QLoRA；够用优先 LoRA，速度和质量更稳。

对应任务：学习舱 `w12d3`（4 个）

☐ DAY 4 训练实战

先跑通小模型，再谈效果；先控制变量，再调参。

核心知识

- 关键参数：learning rate、epoch、batch size、grad accumulation、warmup、weight decay。
- 显存：模型权重 + 梯度 + 优化器状态 + 激活 + KV cache；QLoRA 主要省权重。
- 监控：loss、验证集指标、显存、吞吐、梯度范数。
- 保存：adapter、tokenizer、训练配置、数据版本。
- 推理：加载基座 + adapter，模板/参数与训练一致。

再深入

- 机制：训练是数据、模板、超参、显存和评测的共同作用；loss 只是优化目标。
- 边界：loss 下降可能过拟合；学习率过大会崩；训练/推理不一致会白训。

常见坑

- 只看 training loss，不看验证集和人工样例。
- 学习率过大导致 loss 爆炸/灾难性遗忘。

> **面试口述**
> 训练 loss 下降但效果变差，可能是什么原因？

追问

Q：loss 正常但效果差，先查什么？A：模板、label mask、数据质量、推理参数和验证集分布。

对应任务：学习舱 `w12d4`（3 个）

☐ DAY 5 微调评测：对比 Prompt 基线

微调不是免费午餐，要用同一评测集证明值得。

核心知识

- 对比：base / prompt / RAG / LoRA，固定评测集和解码参数。
- 指标：任务准确率、格式合法率、人工质量、延迟、显存、成本。
- 灾难性遗忘：在通用能力集上回归，和基座对比。
- 过拟合：验证集早停、数据增强、降低 rank/epoch。
- 结论要明确：是否上线、什么场景用、维护成本多少。

再深入

- 机制：微调评测要和 Prompt/RAG 基线对比，固定评测集、解码参数和评测脚本。
- 边界：只在训练分布上测会虚高；通用能力退化是灾难性遗忘；维护成本要算。

常见坑

- 微调后只在训练分布上测，换个问法就崩。
- 不记录训练配置和数据版本，结果无法复现。

> **面试口述**
> 微调后怎么判断有没有灾难性遗忘？

追问

Q：怎么检测灾难性遗忘？A：在通用能力集回归，和基座对比，人工抽检多样输入。

对应任务：学习舱 `w12d5`（4 个）

☐ DAY 6 Lab：小模型微调与对比

一次可复现的实验，比十个“跑通了”的截图有价值。

核心知识

- 固定 seed、数据版本、代码 commit、超参、环境。
- 跑 3 组配置（rank/lr/epoch）对比，选最优。
- 保存 adapter、日志、评测脚本、失败案例。
- 报告结构：任务/数据 → 基线 → 配置 → 结果 → 结论 → 限制。
- 面试时能现场解释每个参数为什么这么选。

再深入

- 机制：可复现实验需要数据版本、代码 commit、超参、seed、环境和评测脚本。
- 边界：不固定 seed 无法比较；只保存最终模型无法复现；数据泄漏会让结论错误。

常见坑

- 不固定 seed，结果每次都不一样，无法比较。
- 只保存最终模型，不保存数据/配置，无法复现。

> **面试口述**
> 用 3 分钟讲你的微调实验。

追问

Q：怎么保证实验可复现？A：固定 seed/环境，记录数据 hash 和配置，脚本一键跑。

对应任务：学习舱 `w12d6`（4 个）

☐ DAY 7 周复盘：微调与数据

能说“不该微调”比会微调更加分。

核心知识

- 能画 Prompt/RAG/Fine-tune 决策树。
- 能解释 LoRA/QLoRA、rank/alpha、显存构成。
- 能说清数据模板、loss mask、灾难性遗忘。
- 能用评测证明微调是否值得。

再深入

- 机制：微调决策树：Prompt → RAG → 微调；能不用微调就不微调。
- 边界：知识频繁变化、数据不足、需要引用/权限时不适合微调；维护成本常被低估。

常见坑

- 把微调当万能药，忽略数据和维护成本。
- 只讲训练，不讲评测和回滚。

> **面试口述**
> 什么情况下你会建议不做微调？

追问

Q：什么情况下建议不做微调？A：Prompt/RAG 已达标、数据质量差、团队无维护能力时。

对应任务：学习舱 `w12d7`（4 个）

W13 推理部署与 Java AI 工程

7 节知识卡 · 先回忆，再对照

☐ DAY 1 vLLM 与推理服务

推理服务的核心指标是吞吐、首 token、显存，不是参数量。

核心知识

- PagedAttention 把 KV cache 分页管理，减少碎片，提高并发。
- 连续批处理动态合并请求，减少 GPU 空等。
- 关键参数：max-model-len、gpu-memory-utilization、max-num-seqs、张量并行。
- OpenAI 兼容 API 方便接入现有客户端。
- 部署要监控：吞吐、P95、显存、排队长度、错误率。

再深入

- 机制：PagedAttention 把 KV cache 分页管理，减少碎片；连续批处理动态合并请求。
- 边界：长上下文吃显存；max-model-len 太大降低并发；量化/并行影响吞吐。

常见坑

- 上下文开太大，KV cache 吃满显存，并发骤降。
- 只看单请求延迟，不看并发吞吐。

> **面试口述**
> vLLM 为什么吞吐高？代价是什么？

追问

Q：vLLM 为什么吞吐高？A：分页 KV cache + 连续批处理 + 高 GPU 利用率。

对应任务：学习舱 `w13d1`（4 个）

☐ DAY 2 量化与性能三角

量化省显存、提吞吐，但必须用业务样本验证质量。

核心知识

- FP16 → INT8/INT4：权重/激活精度下降，显存降低，吞吐提升。
- AWQ/GPTQ 是常见权重量化方法；KV cache 也可量化。
- 质量损失在长尾/推理/代码任务上更明显。
- 量化需要校准数据；校准分布和业务不一致会掉点。
- 选型看质量、显存、吞吐、硬件/算子兼容。

再深入

- 机制：量化降低权重/激活精度，减少显存、提升吞吐；AWQ/GPTQ 是常见权重量化。
- 边界：质量损失在长尾/推理/代码任务更明显；校准数据分布要匹配业务；硬件兼容性。

常见坑

- 只看 perplexity，不看业务评测集。
- 量化后推理参数/模板没变，误以为是模型质量问题。

> **面试口述**
> 量化的收益和风险是什么？怎么验证质量？

追问

Q：量化后质量下降怎么办？A：换更高精度、校准数据、混合精度或只量化部分层。

对应任务：学习舱 `w13d2`（4 个）

☐ DAY 3 Spring AI 基础

Java 不是包袱；企业集成、权限、事务和运维才是 AI 落地的稀缺能力。

核心知识

- ChatClient 提供 fluent API：prompt、stream、structured output、advisors。
- PromptTemplate 管理变量；Advisor 做 RAG、日志、护栏等横切逻辑。
- 流式用 Flux<String> / SSE，注意背压和超时。
- 结构化输出用 JSON schema/实体映射，仍要应用侧校验。
- Spring AI 适合把 AI 接进现有 Spring 微服务体系。

再深入

- 机制：Spring AI 用 ChatClient/Advisor 抽象模型调用、RAG、流式和结构化输出。
- 边界：复杂 agent 生态不如 Python；Java 服务要处理超时/熔断/背压；不要把所有逻辑塞进 Java。

常见坑

- 为了“统一语言”把所有 AI 逻辑塞进 Java，牺牲实验效率。
- 不做超时/降级，模型供应商故障拖垮主站。

> **面试口述**
> 为什么用 Spring AI 而不是再写一个 Python 服务？边界怎么划？

追问

Q：Java 和 Python 服务边界怎么划？A：Java 做企业集成/权限/事务，Python 做实验/Agent 编排。

对应任务：学习舱 `w13d3`（4 个）

☐ DAY 4 Spring AI RAG 与模型网关

把模型供应商当不可靠下游治理，和微服务网关思路一致。

核心知识

- RAG：VectorStore/Retriever + Advisor；引用、权限、租户过滤仍要自己做。
- 模型网关：统一协议、鉴权、配额、路由、熔断、降级、计量。
- 路由维度：任务类型、质量、成本、延迟、可用性。
- 降级：备用模型/小模型/缓存答案；记录降级原因。
- 灰度：按租户/比例分流，监控质量与成本。

再深入

- 机制：模型网关统一协议、鉴权、配额、路由、熔断、降级和计量。
- 边界：路由要有可解释原因；降级要记录；多供应商增加配置和测试成本。

常见坑

- 路由没有可解释原因，线上无法排障。
- 降级后不记录，用户得到低质量答案却无人知晓。

> **面试口述**
> 多模型路由策略怎么设计？如何做灰度？

追问

Q：路由策略怎么设计？A：按任务、质量、成本、延迟、可用性；灰度 + 监控 + 回滚。

对应任务：学习舱 `w13d4`（4 个）

☐ DAY 5 Java 生产化：SSE、背压、熔断、指标

Java 候选人的杀手锏是工程化，不是会调 API。

核心知识

- WebFlux Flux<ServerSentEvent> 做流式；限制缓冲，按下游消费。
- 客户端断开时取消上游，释放连接。
- Resilience4j：超时、重试、熔断、限流、舱壁。
- Micrometer 采集延迟、错误、token/调用计数，接 Prometheus。
- 缓存：精确/语义缓存要带租户和版本。

再深入

- 机制：WebFlux 用 Flux/Mono 表达流式；背压让下游按消费能力拉取；取消向上游传播。
- 边界：无界缓冲会 OOM；超时/断连要清理；熔断要覆盖检索/DB/模型。

常见坑

- 无界缓冲导致 OOM；流式一定要背压。
- 熔断只包模型调用，没包检索/DB，整体仍会挂。

> **面试口述**
> WebFlux 下流式返回怎么处理背压和超时？

追问

Q：怎么处理背压？A：限制缓冲、按请求消费、超时取消、监控队列长度。

对应任务：学习舱 `w13d5`（4 个）

☐ DAY 6 Lab：项目三完整版

项目三证明“你能把 AI 接进企业系统”，不是“你会写 Python”。

核心知识

- 功能：模型网关 + RAG + 流式 + 降级 + 缓存 + 监控。
- 权限：复用现有鉴权/租户/审计，不另起一套。
- 指标：可用性、P95、缓存命中、单次成本。
- 成本表：输入/输出 token、检索、rerank、缓存节省。
- 边界：Python 负责实验/Agent，Java 负责企业集成/稳定性的场景。

再深入

- 机制：项目三证明“Java 能把 AI 接进企业系统”，核心是网关、RAG、流式、降级、监控和成本。
- 边界：只调 API 会和项目一重复；没有压测/成本表无法证明生产化。

常见坑

- 只做“调用模型”，没有网关/降级/监控，和项目一重复。
- 压测数据没有环境说明，不可信。

> **面试口述**
> 用 10 分钟讲项目三，和项目一的区别是什么？

追问

Q：为什么不用 Python？A：复用 Java 生态的鉴权/租户/事务/监控，边界按场景划分。

对应任务：学习舱 `w13d6`（4 个）

☐ DAY 7 周复盘 + 项目三模拟面试

把“Java 不是包袱，是落地能力”讲成故事。

核心知识

- 高频追问：为什么不用 Python？路由/降级？SSE 背压？缓存一致性？成本？
- 每个机制要有数据和失败案例。
- 与现有权限体系怎么集成，是企业面试的重点。
- 项目三的边界要清楚，不贬低 Python。

再深入

- 机制：Java 候选人的差异化是工程化 + 企业集成，不是和算法候选人拼训练。
- 边界：不要贬低 Python；要能说清边界和取舍；没有数据支撑会显得空。

常见坑

- 把项目三讲成“Spring AI 学习 demo”，没有企业问题。
- 指标只有 QPS，没有可用性/成本。

> **面试口述**
> 项目三最容易被追问什么？

追问

Q：项目三最容易被追问什么？A：为什么 Java、路由/降级、SSE 背压、缓存一致性、成本。

对应任务：学习舱 `w13d7`（3 个）

W14 AI 系统设计、ML 八股与算法

7 节知识卡 · 先回忆，再对照

☐ DAY 1 系统设计框架与 RAG 设计

先对齐需求与约束，再画架构；不要一上来堆组件。

核心知识

- 回答结构：功能/非功能 → 规模估算 → 数据模型 → 核心链路 → 瓶颈 → 取舍。
- 先问三件事：数据规模/类型、用户/租户/权限、质量与延迟/成本 SLO。
- RAG 核心：ingestion、检索、rerank、生成、引用、评测、观测。
- 容量：QPS × token × 单价；峰值、缓存、冗余。
- 失败模式：召回漏、上下文噪声、模型不用证据、越权、成本失控。

再深入

- 机制：系统设计先对齐功能/非功能、规模、数据模型和 SLO，再画架构。
- 边界：不估规模会选错存储；不问权限会埋越权；只画正常路径会被追问故障。

常见坑

- 直接画向量库/模型，没问数据规模和权限。
- 只讲正常路径，不讲失败/降级/成本。

> **面试口述**
> 设计企业 RAG 平台，先问哪三个问题？

追问

Q：先问哪三个问题？A：数据规模/类型、用户/租户/权限、质量/延迟/成本 SLO。

对应任务：学习舱 w14d1（4 个）

☐ DAY 2 Agent 平台与模型网关设计

Agent 平台的核心是状态、工具、权限、审批和观测，不是多智能体。

核心知识

- Agent 平台：API、编排引擎、工具运行时、状态存储、trace、评测。
- 工具协议：schema、超时、错误、幂等、权限、沙箱。
- 长任务：队列、checkpoint、恢复、取消、超时。
- 模型网关：统一协议、路由、配额、熔断、降级、计量。
- 安全：工具白名单、人工审批、审计、租户隔离。

再深入

- 机制：Agent 平台 = API + 编排引擎 + 工具运行时 + 状态存储 + trace + 评测。
- 边界：工具权限、状态持久化、审批、死循环和成本是核心难点；多 agent 不是默认。

常见坑

- 把 Agent 平台设计成“prompt 管理器”，没有状态和工具治理。
- 网关只做转发，没有配额/熔断/降级。

> **面试口述**
> 设计 Agent 平台的核心难点是什么？

追问

Q：Agent 平台最难的是什么？A：可控性：权限、状态、审批、观测和评测。

对应任务：学习舱 w14d2（4 个）

☐ DAY 3 Transformer 与 ML 八股

八股要能一句话说清，再展开一层；不要背长段落。

核心知识

- Attention：QKV、缩放、多头、位置编码、复杂度 $O(n^2)$。
- 归一化：LayerNorm 逐样本特征维，适合变长序列；BatchNorm 依赖 batch。
- 偏差-方差：欠拟合/过拟合；正则、早停、更多数据、简化模型。
- 评估：准确率/ precision/recall/F1/AUC；类别不平衡看 PR/F1。
- 过拟合信号：训练好、验证差；数据泄漏会让分数虚高。

再深入

- 机制：Transformer 的核心是 attention + FFN + 残差 + 归一化；LayerNorm 逐样本在特征维归一化。
- 边界：BatchNorm 依赖 batch，变长序列不稳定；attention 是 $O(n^2)$；过拟合/欠拟合要看训练/验证差距。

常见坑

- 只背公式，不能解释“为什么这样设计”。
- 把 LayerNorm/BatchNorm 的维度说反。

> **面试口述**
>
> LayerNorm 和 BatchNorm 的区别？为什么 Transformer 常用 LayerNorm？

追问

Q：LayerNorm 和 BatchNorm 的区别？A：逐样本特征维 vs batch 维；Transformer 常用 LayerNorm。

对应任务：学习舱 `w14d3`（4 个）

☐ DAY 4 算法模板：8 类高频模式

刷题不是比数量，是看到题能反射出模式。

核心知识

- 哈希：计数、去重、前缀和配合。
- 双指针：有序数组、去重、三数之和。
- 滑动窗口：连续子数组/子串的最长/最短。
- 二分：有序、答案单调、边界查找。
- 堆：前 K 个、合并 K 个、流式中位数。

再深入

- 机制：算法题先识别模式：哈希、双指针、滑窗、二分、堆、图、DP、回溯。
- 边界：连续子数组看滑窗/前缀和/DP；前 K 个看堆/快速选择；依赖顺序看拓扑。

常见坑

- 只写代码不分析复杂度，面试官会追问。
- 不处理空输入/边界/重复元素。

> **面试口述**
>
> 看到“连续子数组/子串”优先想什么？看到“前 K 个”呢？

追问

Q：看到连续子数组/子串优先想什么？A：滑窗；有负数区间和可考虑前缀和/DP。

对应任务：学习舱 `w14d4`（4 个）

☐ DAY 5 项目三系统设计与容量

面试官爱问“10 倍”，答案要有数字和顺序。

核心知识

- 容量估算：QPS、token、并发、存储、带宽、成本。
- 瓶颈顺序：上游配额/限流 → 连接池 → DB → 缓存 → CPU/GPU。
- 扩展：缓存、队列、批处理、分片、读写分离、降级。
- 容灾：供应商故障、超时、限流、区域故障。
- 成本：模型分级、缓存、上下文裁剪、批处理。

再深入

- 机制：容量估算按 QPS × token × 单价，考虑峰值、缓存、冗余和降级。
- 边界：模型配额、连接池、DB、缓存和成本都可能先成为瓶颈；只加机器不一定买到 token。

常见坑

- 只说“加机器”，没有瓶颈分析。
- 忽略模型配额和成本，扩容也买不到 token。

> **面试口述**
>
> 流量涨 10 倍，项目三哪里先崩？怎么扩展？

追问

Q：10 倍流量先崩哪？A：上游配额/限流、连接池、DB、缓存，按依赖排序。

对应任务：学习舱 `w14d5`（3 个）

☐ DAY 6 全真模拟面试 1

模拟面试不是考知识，是练结构和抗压。

核心知识

- 45 分钟项目 + 45 分钟系统设计 + 15 分钟行为。
- 项目：背景→方案→取舍→指标→失败。
- 系统设计：需求→估算→架构→瓶颈→取舍。
- 行为：STAR，结果有数字。
- 录像复盘：卡顿、跑题、超时、被追问慌。

再深入

- 机制：模拟面试检验结构、时间控制和抗压；录像回听比自我感觉可靠。
- 边界：只练知识不练表达会超时/跑题；只看对错不看结构会重复犯错。

常见坑

- 只练知识，不练表达和时间控制。
- 复盘只看“对不对”，不看结构。

> **面试口述**
>
> 模拟面试最大的问题通常是什么？

追问

Q：模拟面试最大问题是什么？A：结构乱、超时、只讲技术不讲取舍。

对应任务：学习舱 `w14d6`（3 个）

☐ DAY 7 周复盘：弱项补洞

最后两周不开新坑，只补最影响面试的。

核心知识

- 按“影响 × 频率”排序弱项。
- 每个弱项：一份好材料 → 一页提纲 → 口述/做题 → 反馈 → 再讲。
- 系统设计/项目指标/行为故事是高频失分点。
- 保持算法手感，但不要刷难题找挫败。

再深入

- 机制：最后两周按“影响 × 频率”补弱项，不再开新坑。
- 边界：从第一章系统学来不及；追求全部学会会拖延面试。

常见坑

- 继续收集课程，不输出。
- 追求“全部学会”，导致面试延期。

> **面试口述**
> 你目前最需要补的三个面试弱项？

追问

Q：怎么选三个弱项？A：面试反馈里高频、影响通过率、可在两周内练会的。

对应任务：学习舱 `w14d7`（4 个）

W15 简历、项目包装与模拟面试

7 节知识卡 · 先回忆，再对照

☐ DAY 1 简历 v3：一页、三条故事线、数字

简历不是经历清单，是“你能解决什么问题”的证据。

核心知识

- 一页；3 个项目，每个 3–4 条 bullet。
- bullet 结构：动作 + 技术 + 难点 + 可验证结果。
- 保留 Java 高并发/微服务亮点，这是差异化。
- 关键词对齐 JD：RAG、Agent、pgvector、RAGAS、vLLM、Spring AI。
- 每个数字要能解释口径和来源。

再深入

- 机制：简历是证据链：动作 + 技术 + 难点 + 可验证结果；一页聚焦 3 个项目。
- 边界：指标要真实可解释；堆“熟悉”没有说服力；Java 亮点不要丢。

常见坑

- 堆“熟悉/了解”，没有项目和结果。
- 指标造假；面试官一追问就露馅。

> **面试口述**
> 你的简历最容易被质疑哪一条？

追问

Q：最容易被质疑哪条？A：指标口径和“是不是你做的”；准备数据来源和失败复盘。

对应任务：学习舱 `w15d1`（3 个）

☐ DAY 2 三个项目故事卡：3/10/30 分钟版本

故事卡不是背稿，是确保关键点不漏。

核心知识

- 3 分钟：问题 → 约束 → 架构 → 2 个取舍 → 1 个指标 → 1 个失败。
- 10 分钟：展开链路、数据、替代方案。
- 30 分钟：深入细节、故障、扩展、复盘。
- 每个项目准备 10 个追问：为什么不用 X？失败过什么？指标怎么来？
- 用 STAR 讲行为故事，结果有数字。

再深入

- 机制：3/10/30 分钟版本对应不同信息密度；核心是问题、约束、取舍、指标和失败。
- 边界：3 分钟报菜名会被打断；只讲成功不真实；没有数字无法证明。

常见坑

- 3 分钟版讲成 10 分钟，面试官打断。
- 只讲成功，不讲失败和取舍。

> **面试口述**
> 3 分钟讲项目，结构是什么？

追问

Q：3 分钟项目介绍结构？A：问题→约束→架构→2 个取舍→1 个指标→1 个失败。

对应任务：学习舱 `w15d2`（3 个）

☐ DAY 3 作品集与 demo

先让外行 3 分钟看懂，再让内行 30 分钟追问。

核心知识

- GitHub README：问题、架构、启动、指标、取舍、限制。
- 演示视频：问题→操作→结果→指标。
- 在线 demo 可选；至少保证本地一键启动。
- 仓库无密钥、无大文件、commit 清晰。
- 作品集首页：3 个项目 + 简历 + 联系方式。

再深入

- 机制：作品集让面试官快速判断“能跑、能讲、可信”；README 是第一入口。
- 边界：仓库跑不起来会直接减分；密钥/内部数据泄露是红线；视频太长没人看。

常见坑

- 仓库跑不起来，README 只有安装命令。
- 提交了 API key/内部数据。

> **面试口述**
> 面试官点开你的 GitHub，最想看到什么？

追问

Q：GitHub 最想看到什么？A：可运行、架构清晰、指标可信、无密钥、有失败复盘。

对应任务：学习舱 `w15d3`（3 个）

☐ DAY 4 模拟面试 2：系统设计 + 项目深挖

面试不是辩论，是展示权衡过程。

核心知识

- 系统设计：需求→估算→架构→瓶颈→取舍。
- 被问“为什么不用 X”：比较 2–3 个维度，说明当前约束，给出何时会换。
- 项目深挖：指标口径、失败、替代方案。
- 时间控制：先给结论，再展开。
- 录像复盘：结构、语速、卡顿、跑题。

再深入

- 机制：系统设计面试看权衡过程，不是标准答案；先说结论再展开。
- 边界：防御式回答“X 不好”会扣分；超时会没时间讲取舍；没有数字显得空。

常见坑

- 防御式回答“X 不好”，而不是比较。
- 超时，最后没时间讲取舍。

> **面试口述**
> 系统设计被追问“为什么不用 X”怎么答？

追问

Q：怎么答替代方案？A：承认替代方案，比较 2–3 个维度，给出当前约束和切换条件。

对应任务：学习舱 `w15d4`（3 个）

☐ DAY 5 行为面试、转型故事与首批投递

首批投递是练手，不要等“准备好”才投。

核心知识

- 30 秒转型故事：动机→行动→差异化→目标。
- 3 个 STAR：冲突、失败、主动性。
- 离职动机：不贬低前公司，讲成长与匹配。
- 薪资：给区间，讲总包和匹配度。
- 投递记录：公司/岗位/渠道/日期/反馈。

再深入

- 机制：30 秒转型故事 = 动机→行动→差异化→目标；行为故事用 STAR。
- 边界：贬低 Java 会显得不专业；编造故事会被追问击穿；首批投递是练手。

常见坑

- 把转型故事讲成“Java 没前途”。
- 首批投递只看大厂，反馈周期长。

> **面试口述**
> 为什么从 Java 转 AI？30 秒版。

追问

Q：为什么从 Java 转 AI？A：看到落地瓶颈在工程化，补 AI 能力，用 Java 工程化做差异化。

对应任务：学习舱 `w15d5`（4 个）

☐ DAY 6 春节：最低完成线 + 休息

休息一天不会影响进度，连续性比单日强度重要。

核心知识

- 春节只做轻复盘：看一遍项目故事卡，口述 1 分钟。
- 写 3 行节后计划，不做硬任务。
- 睡眠/陪伴/运动也是长期学习的一部分。
- 节后按最低线接上，不要补作业式焦虑。

再深入

- 机制：春节最低线是保持连续性，不是追求进度；休息也是计划的一部分。
- 边界：彻底断掉会提高节后启动成本；疯狂补课会倦怠。

常见坑

- 假期彻底断掉，节后启动成本很高。
- 假期疯狂补课，节后倦怠。

> **面试口述**
> 休息一天会影响进度吗？

追问

Q：休息一天会影响进度吗？A：不会；长期计划允许弹性，连续性比单日强度重要。

对应任务：学习舱 `w15d6`（3 个）

☐ DAY 7 恢复 + 模拟面试 3 + 投递复盘

没有回音是数据，不是否定；用数据改下一步。

核心知识

- 投递漏斗：投递→沟通→一面→二面→终面→offer。
- 没回音先看匹配/关键词/渠道/内推，不要只改措辞。
- 模拟面试 3：行为 + 3 分钟项目，录像。
- 调整目标公司分档：冲刺/匹配/保底。
- 节后重启：先投 10 家，再根据反馈补洞。

再深入

- 机制：投递漏斗诊断：投递→沟通→一面→二面→终面→offer，每层转化率指向不同问题。
- 边界：没回音不一定是能力问题，可能是匹配/渠道/时机；只改简历不够。

常见坑

- 投 10 家没回音就否定自己。
- 只改简历，不换渠道/不找内推。

> **面试口述**
> 投了 10 家没回音，先改什么？

追问

Q：漏斗怎么诊断？A：沟通少看简历/匹配，一面少看基础/表达，二面少看项目深度。

对应任务：学习舱 `w15d7`（3 个）

W16 投递、面试迭代与 Offer

7 节知识卡 · 先回忆，再对照

☐ DAY 1 定向投递 20 家 + 内推

投递量是概率游戏，但定向比海投有效。

核心知识

- 目标公司分三档：冲刺 / 匹配 / 保底。
- 优先有真实 AI 落地场景的团队：知识库、客服、合同、报表、营销。
- 内推提高曝光，不保证录取；找前同事/脉脉/社区。
- 每个岗位记录 JD 关键词、渠道、日期、跟进时间。
- 简历按岗位做小幅定制，不伪造经历。

再深入

- 机制：目标公司分冲刺/匹配/保底，结合 AI 落地场景、团队阶段和技术栈匹配。
- 边界：只投大厂反馈慢；海投同一份简历命中率低；内推提高曝光但不保证录取。

常见坑

- 只投大厂，反馈慢、竞争大。
- 海投同一份简历，关键词不匹配。

> **面试口述**
> 怎么选目标公司？怎么分档？

追问

Q：怎么选目标公司？A：看落地场景、阶段、匹配、稳定性、成长和通勤，分档控制风险。

对应任务：学习舱 w16d1（3 个）

☐ DAY 2 面试复盘与弱项补洞

复盘不是记流水账，是找可复用的模式。

核心知识

- 24 小时内复盘：问题、回答、卡点、更好结构、追问、下次动作。
- 分类：不会 / 不熟 / 表达差；不同问题不同对策。
- 知识洞补 1 页提纲；表达洞练 3 次口述。
- 同一类问题连续出现，优先解决。
- 更新速答卡和错题本。

再深入

- 机制：复盘 24 小时内完成，按“不会/不熟/表达差”分类，分别补。
- 边界：只记流水账找不到模式；每个问题都补会导致一个都没补深。

常见坑

- 面试完只记“感觉不好”，没有具体问题。
- 每个问题都补，结果一个都没补深。

> **面试口述**
> 面试复盘要记什么？为什么强调 24 小时？

追问

Q：复盘重点是什么？A：可复用的模式和下一次的具体动作，不是情绪。

对应任务：学习舱 w16d2（3 个）

☐ DAY 3 模拟面试 3 + 压力测试

面试官更看重思路，不是你什么都会。

核心知识

- 压力测试：连续追问、打断、质疑数据。
- 不会的问题：承认→拆解→假设→验证方式。
- 保持结构：先结论，再原因/取舍。
- 情绪管理：放慢语速，先复述问题。
- 录像复盘：表情、语速、填充词、跑题。

再深入

- 机制：压力面试看的是结构和诚实，不是“什么都会”。
- 边界：编数据会被追问击穿；沉默太久会显得没思路。

常见坑

- 编数据/硬撑，面试官一追问就崩。
- 沉默太久，或直接说“不会”就结束。

> **面试口述**
> 遇到完全不会的问题怎么答？

追问

Q：完全不会怎么答？A：承认没做过，拆解问题，给出假设和验证路径，不编造。

对应任务：学习舱 w16d3（3 个）

☐ DAY 4 系统设计与八股补洞

面试期学习要“功利”，按反馈补，不按教材顺序。

核心知识

- 只补高频且影响大的：缓存一致性、队列、评测指标、容量估算。
- 每个弱项：1 份好材料 → 1 页提纲 → 口述/做题 → 反馈。
- 算法保持手感：每周 2–3 次，medium 为主。
- 系统设计练“10 倍”和故障场景。
- 错题加入复习队列。

再深入

- 机制：两周补洞按影响×频率排序，输出驱动：一页提纲 + 一次口述/做题。
- 边界：从第一章系统学来不及；只输入不输出面试还是讲不出。

常见坑

- 从第一章系统学，时间不够。
- 只输入不输出，面试还是讲不出。

> **面试口述**
> 怎么在两周内补一个弱项？

追问

Q：怎么两周补一个弱项？A：一份好材料→一页提纲→口述/做题→反馈→再讲一遍。

对应任务：学习舱 w16d4（3 个）

☐ DAY 5 Offer 评估与谈判

不要只比 base；总包确定性、成长和风险都要算。

核心知识

- 总包：base + 奖金 + 股票/期权 + 补贴；注意确定性和兑现条件。
- 岗位：汇报线、团队规模、AI 项目阶段、数据/算力权限。
- 风险：试用期、加班、业务稳定性、裁员历史。
- 谈判：聚焦岗位匹配和市场区间，保持合作态度。
- 决策表：给每个维度权重，避免情绪决策。

再深入

- 机制：Offer 比较看总包确定性、成长、汇报线、稳定性、通勤和试用期，不只看 base。
- 边界：股票/期权有兑现条件；编造竞争 offer 有风险；谈判要专业。

常见坑

- 只看月薪，忽略奖金/股票兑现条件。
- 编造竞争 offer，被拆穿。

> **面试口述**
> 怎么比较两个 offer？谈判怎么开口？

追问

Q：怎么比较两个 offer？A：总包/确定性/成长/稳定性/通勤/试用期，按权重打分。

对应任务：学习舱 `w16d5`（3 个）

☐ DAY 6 16 周复盘与 90 天成长计划

拿到 offer 不是终点，是下一段成长的起点。

核心知识

- 16 周复盘：做成了什么、没做到什么、最大变化、下一步。
- 能力盘点：Python/LLM/RAG/Agent/部署/表达，各打 1–5 分。
- 90 天计划：技术深度、业务理解、英语、输出、人脉。
- 保持节奏：每周算法/项目复述/投递/复盘。
- 把复盘写成面试可用的“最近学习”故事。

再深入

- 机制：16 周复盘看能力变化和系统，不只看学了多少课；90 天计划要可执行。
- 边界：计划太大无法坚持；拿到 offer 就停止学习会入职被动。

常见坑

- 拿到 offer 就停止学习，入职后被动。
- 计划太大，无法坚持。

> **面试口述**
> 16 周最大的变化是什么？

追问

Q：16 周最大变化？A：形成“设计-实现-评测-部署-表达”的系统能力。

对应任务：学习舱 `w16d6`（3 个）

☐ DAY 7 毕业：保持手感，持续投递

计划结束，求职继续；建立每周维护节奏。

核心知识

- 检查漏斗：投递/沟通/一面/二面/终面/offer。
- 每周固定：2 次算法、1 次项目复述、5 家投递、1 次复盘。
- 跟踪一个技术方向，持续输出。
- 面试反馈驱动补洞，不再从零学新课。
- 保持健康：睡眠、运动、社交。

再深入

- 机制：计划结束后的竞争力来自每周固定节奏：算法、项目复述、投递、复盘。
- 边界：拿到 offer 后停摆会退化；没 offer 焦虑到不投递会断循环。

常见坑

- 拿到 offer 后完全停摆。
- 没有 offer 就焦虑到不投递。

> **面试口述**
> 计划结束后怎么保持竞争力？

追问

Q：怎么保持手感？A：每周固定节奏 + 持续输出 + 面试反馈驱动补洞。

对应任务：学习舱 `w16d7`（3 个）
